



Tsinghua University

Modern Computer Architecture Fall 2025

Lecture 03: Basic MIPS Architecture Review

Shuwen Deng

shuwend@tsinghua.edu.cn

2025-Oct-11

Outline

- Review
- Single Cycle Processor
- Multiple Cycle Processor
- Pipeline Processor
- Conclusion

Review: Power within a Processor

- Dynamic power required per transistor:

$$P_{\text{dynamic}} = \frac{E}{\Delta T} = E_{0-1} f_{0-1} = \frac{1}{2} C_L V_{\text{dd}}^2 f_{0-1}$$

- Dynamic power in a microprocessor:

$$P_{\text{dynamic}} = \frac{1}{2} \alpha C_{\text{tot}} V_{\text{dd}}^2 f$$

Switching factor **Total load capacitance** **Frequency**

Review: Static Power

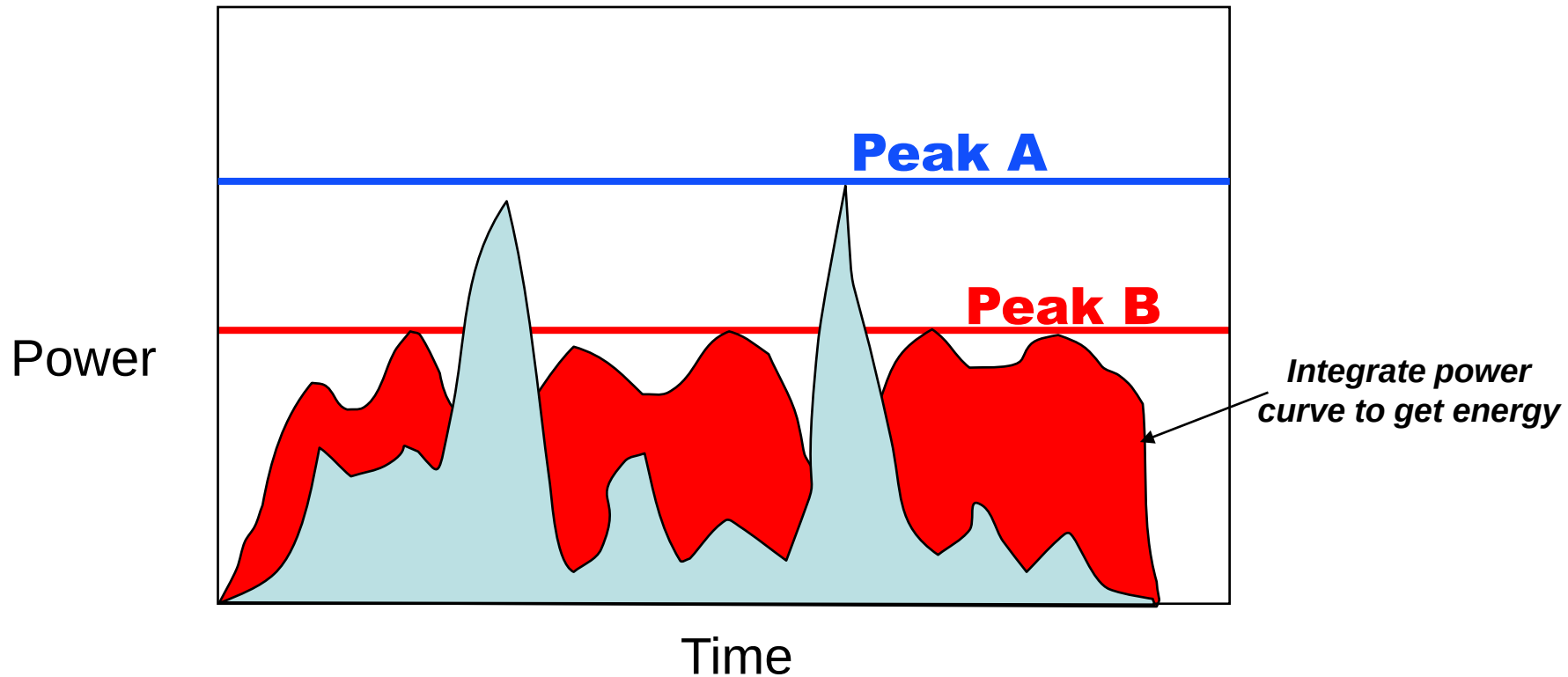
- Static power within a microprocessor:

$$P_{\text{static}} \propto I_{\text{static}} V_{\text{dd}}$$

- Proportional to number of devices
- Leakage can be up to 50% in SRAM

$$I_{\text{subn}} = A_s \frac{W}{L} (v_T)^2 (1 - e^{-V_{DS}/v_T}) e^{(V_{GS} - V_{th})/nv_T}$$

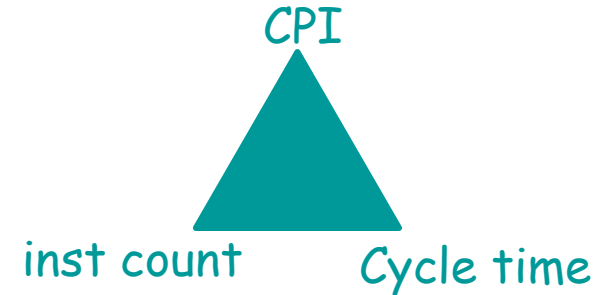
Peak Power versus Average Power



- Peak Power: rush current, reliability
- Average Power: energy, thermal, TDP

Review: The Performance Equation

- Performance equation is then



$$\text{CPU time} = \text{Instruction_count} \times \text{CPI} \times \text{clock_cycle}$$

- These equations separate the **three key** factors that affect performance
 - CPU execution time by running the program
 - Instruction count by using profilers/simulators without knowing all of the implementation details
 - CPI varies by instruction type and ISA implementation

Computer Performance

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

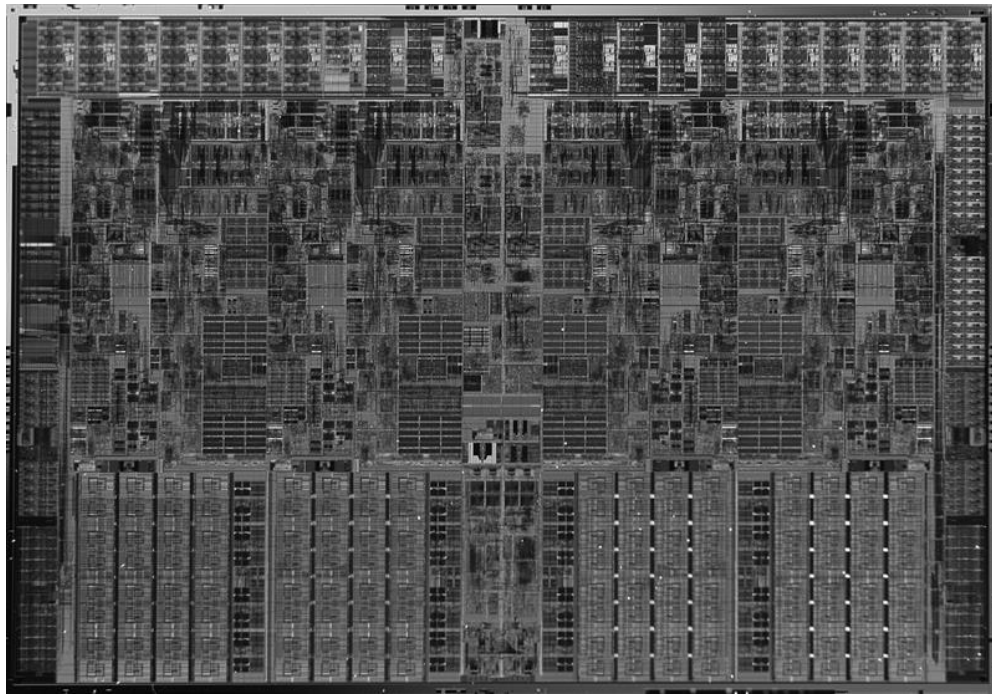
	Inst Count	CPI	Clock Rate
Program	X		
Compiler	X	(X)	
Inst. Set.	X	X	
Organization		X	X
Technology			X

Cost & Yield Conclusion

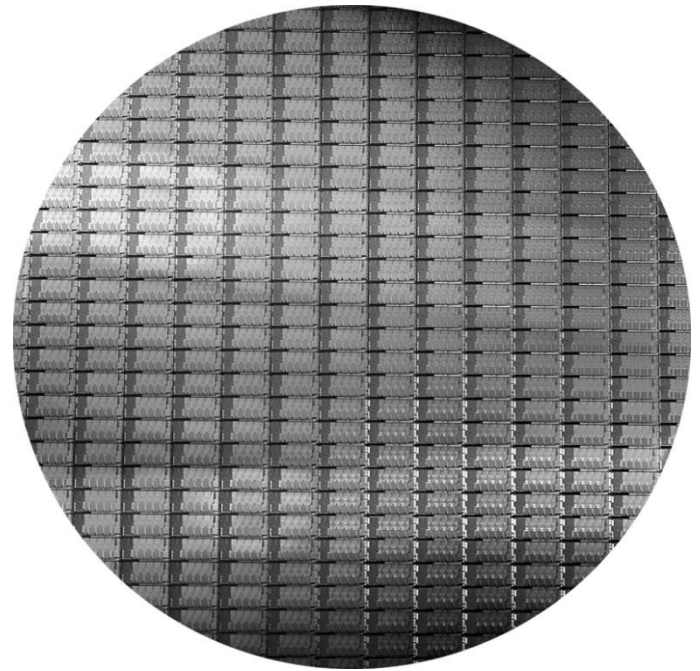
- Cost
 - Yield vs Die Size
- Reliability
 - Thermal issues
 - NBTI etc.
 - MTTF by redundancy

Cost of an Integrated Circuit

- Integrated circuit cost is a great portion of the computer cost



Photograph of an Intel Core i7 microprocessor die, which is evaluated in Chapters 2 through 5. The dimensions are 18.9 mm by 13.6 mm (257 mm²) in a 45 nm process. (Courtesy Intel.)

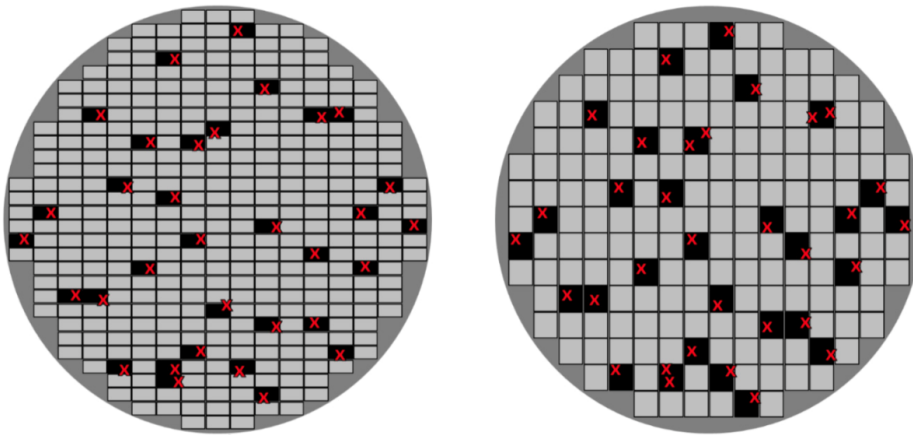


This 300 mm wafer contains 280 full Sandy Bridge dies, each 20.7 by 10.5 mm in a 32 nm process. (Sandy Bridge is Intel's successor to Nehalem used in the Core i7.) At 216 mm², the formula for dies per wafer estimates 282. (Courtesy Intel.)

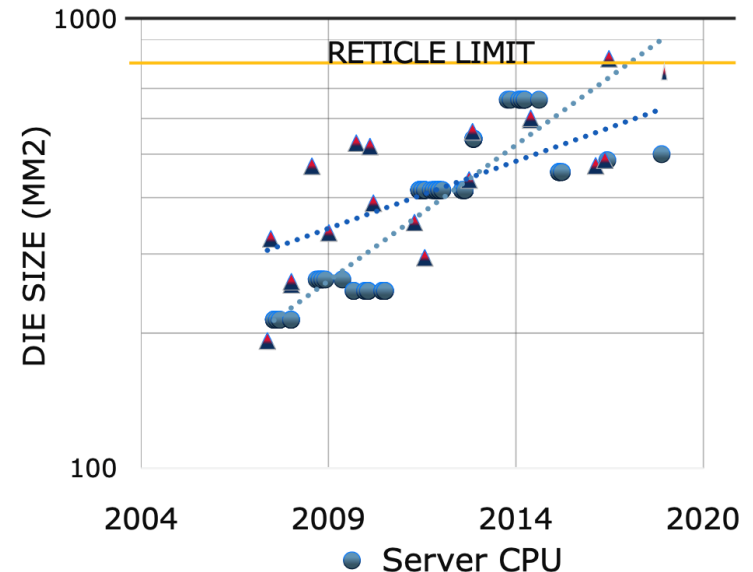
AMD Processor

Bigger Chips to Offset Technology Slowdown

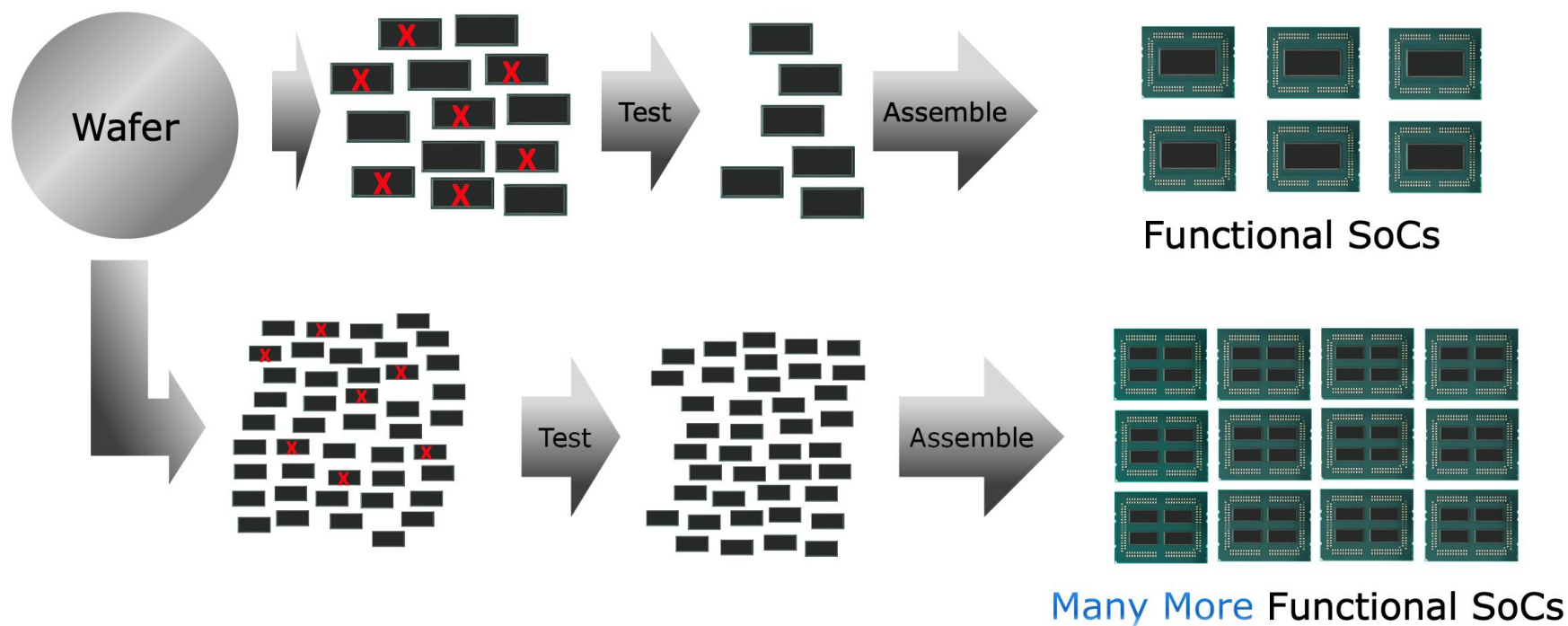
If technology scaling only gives you (say) 1.5x more devices per 24 months, why not just make chips 1.33x bigger to get 2x transistors?



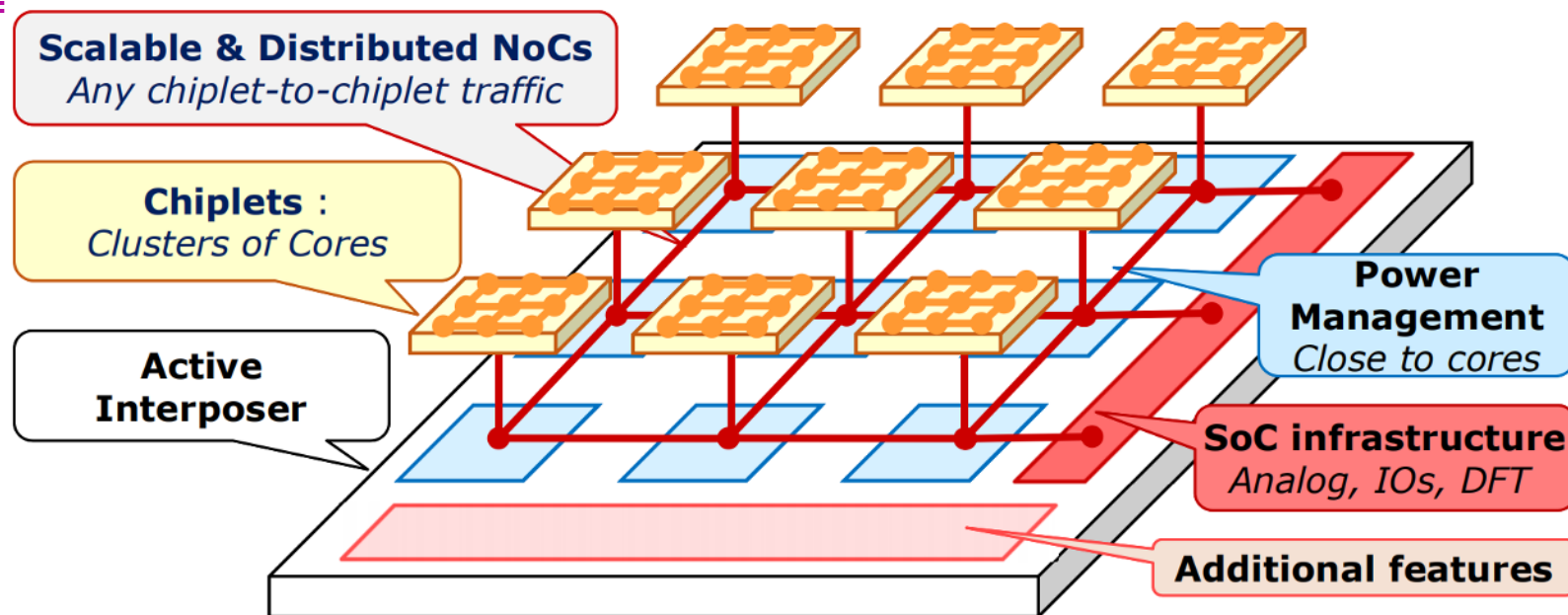
395 chips $\rightarrow$ 362 good die (8% yield loss)
192 chips $\rightarrow$ 162 good die (16% yield loss)
(hypothetical/academic example [1], not real yield rates)



High-Level Approach to Chiplets

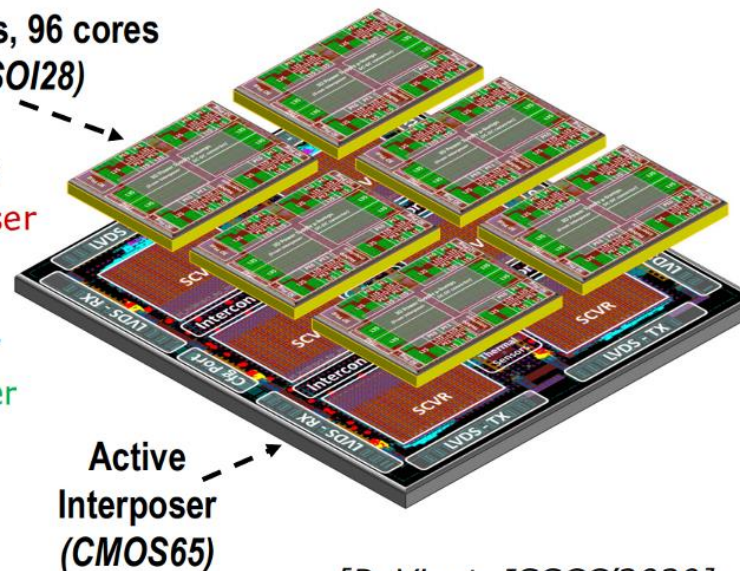


Active interposer



6 Chiplets, 96 cores
(FDSOI28)

- 3 Distributed flexible interconnects
 - Passive connections on interposer between multisynchronous NoCs in chiplets
 - Synchronous NoC on interposer
 - Asynchronous NoC on interposer



[P. Vivet, ISSCC'2020]

清华大学

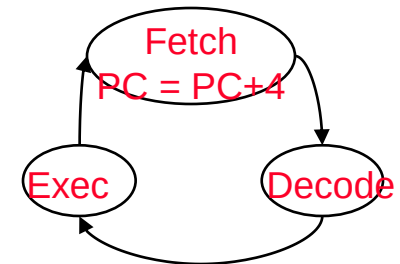
Tsinghua University

Outline

- Review
- Single Cycle Processor
- Multiple Cycle Processor
- Pipeline Processor
- Conclusion

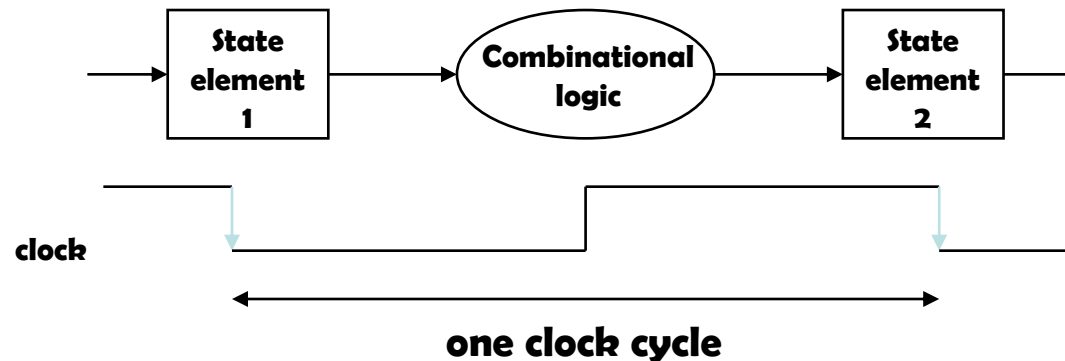
The Processor: Datapath & Control

- Simplified MIPS ISA
 - memory-reference instructions: **lw, sw**
 - arithmetic-logical instructions: **add, sub, and, or, slt**
 - control flow instructions: **beq, j**
- Generic implementation
 - program counter (PC) to supply address
 - fetch the instruction
 - decode the instruction (and read registers)
 - execute the instruction



Clocking Methodologies

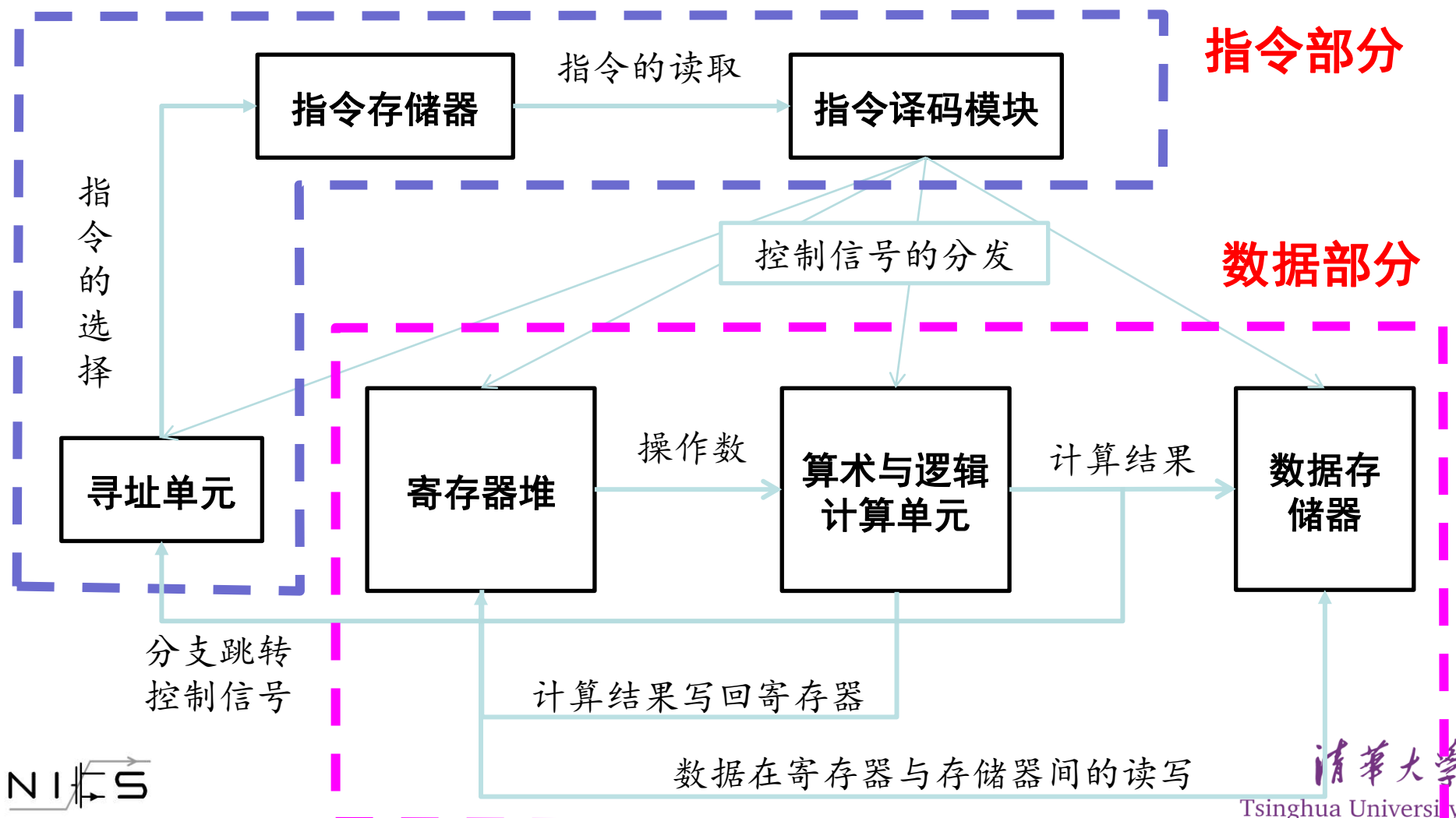
- The clocking methodology: when signals can be read and written
 - Edge-triggered methodology
- Typical execution
 - read contents of state elements
 - send values through combinational logic
 - write results to one or more state elements



- State elements are written on every clock cycle; if not, need explicit write control signal
 - write only when both write control is asserted and clock edge occurs

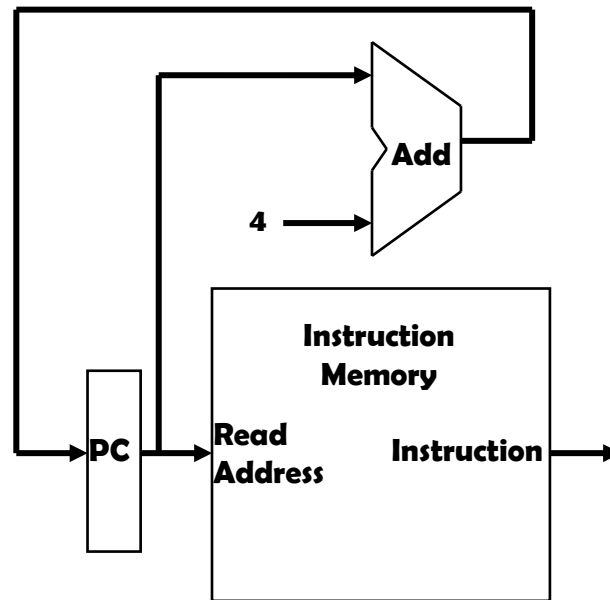
处理器的架构

数据通路



Fetching Instructions

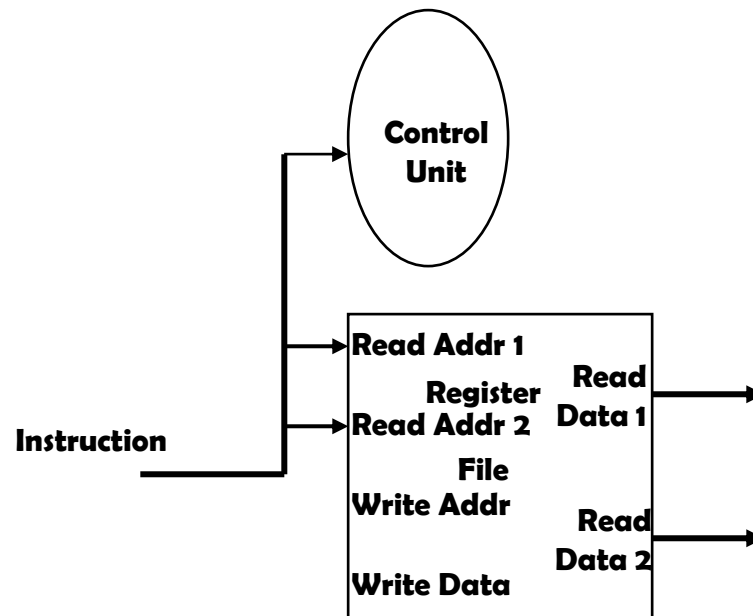
- Fetching instructions involves
 - Reading the instruction from IM
 - Updating the PC to hold the address of the next instruction



- PC is updated every cycle, no need explicit write control
- IM is read every cycle, no need explicit read control

Decoding Instructions

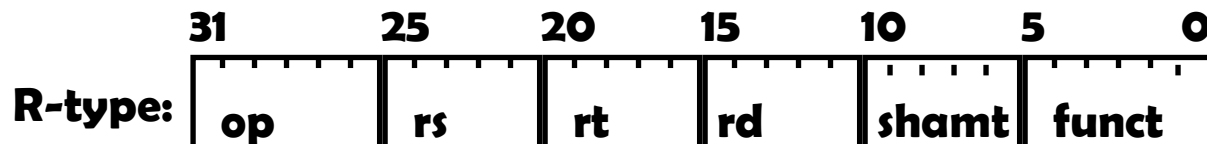
- Decoding instructions involves
 - Sending **opcode** and **function field** to the control unit



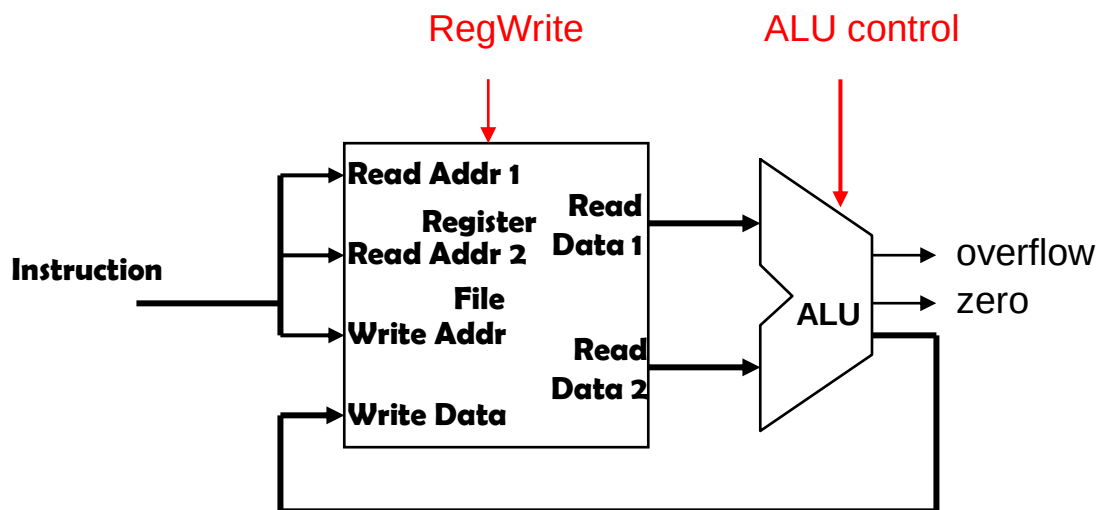
- Reading two values** from the Register File
 - Register File addresses are contained in the instruction

Executing Operations I

- R format operations (**add**, **sub**, **slt**, **and**, **or**)



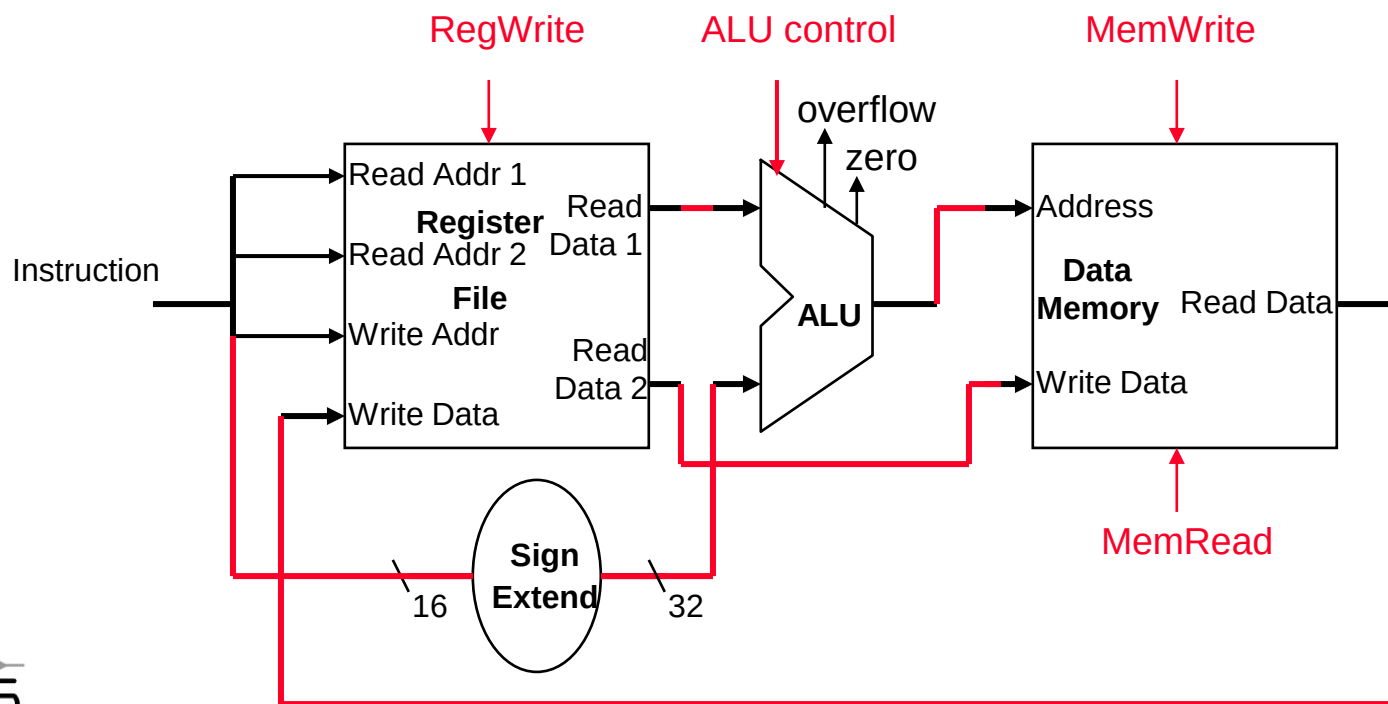
- perform the (**op** and **funct**) operation on values in **rs** and **rt**
- store the result back into location **rd**



- We need an **explicit write control signal** for the Register File

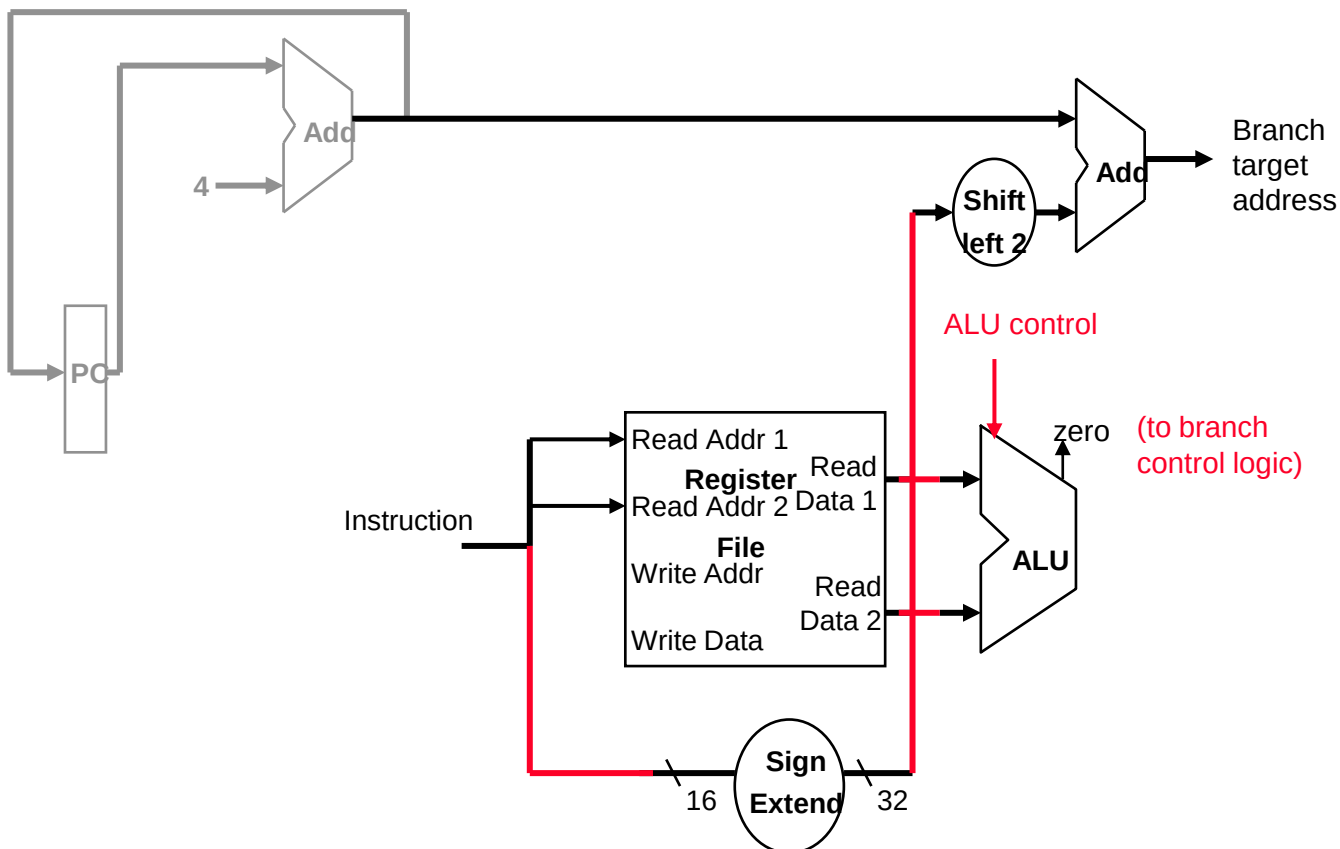
Executing Operations II

- Load and store operations
 - **compute memory address**
 - adding the base register to the 16-bit sign-extended offset field in the instruction
 - **store** value (read from the Register File during decode) written to the Data Memory
 - **load** value, read from the Data Memory, written to the Register File



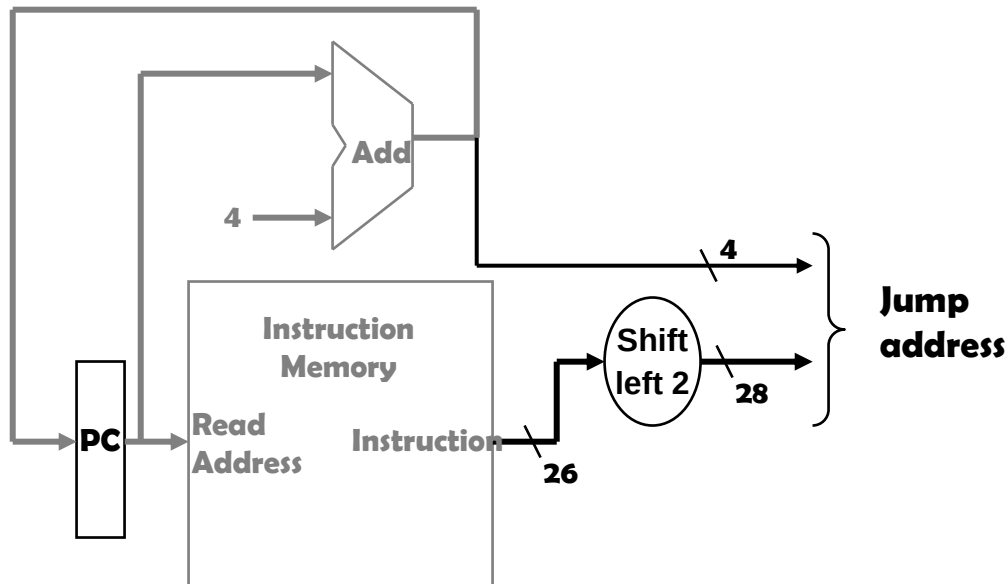
Executing Operations III

- Branch operations involves
 - compare for equality (**zero** ALU output)
 - compute the branch target address by adding the updated PC to the 16-bit signed-extended offset field in the instr



Executing Operations IV

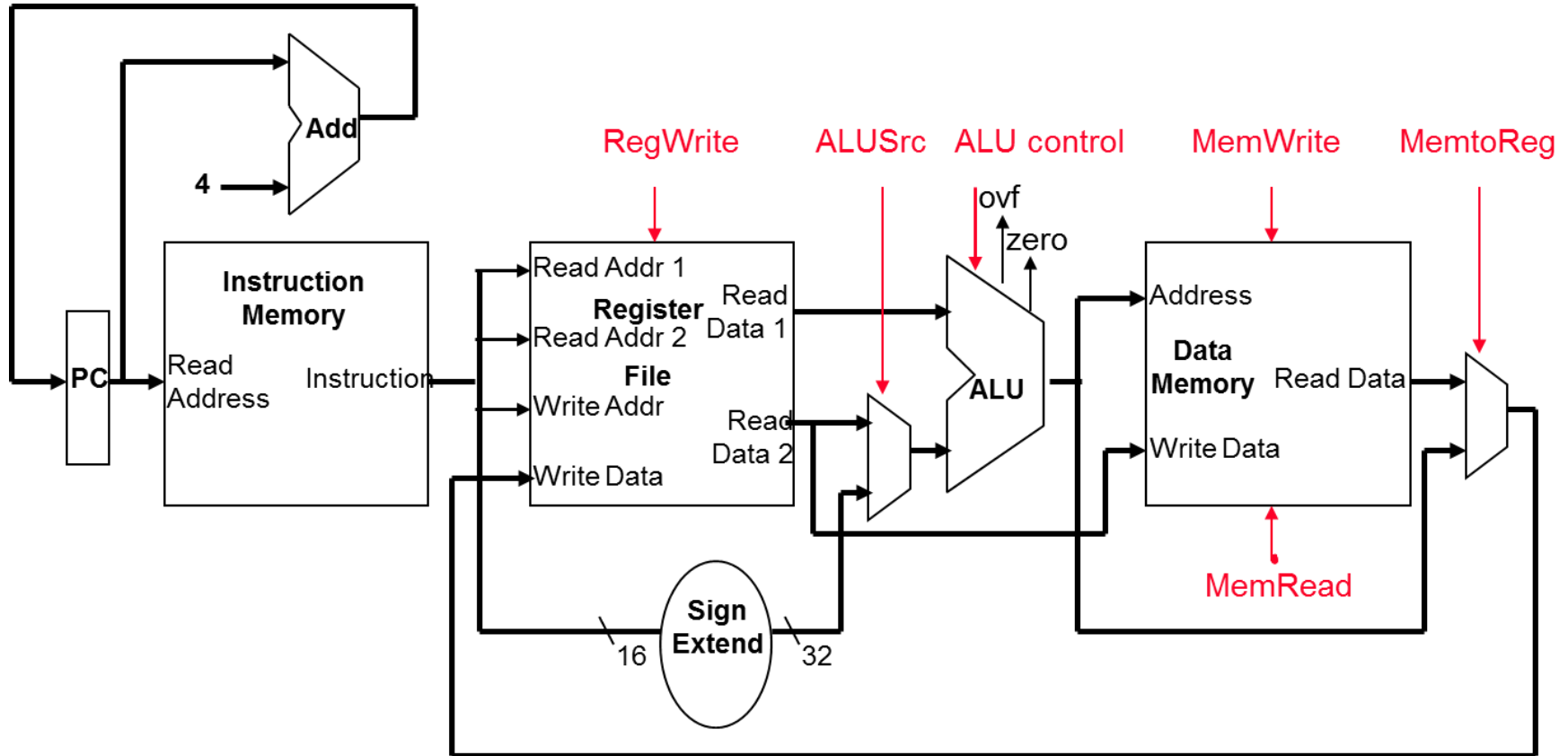
- Jump operation involves
 - Lower 28 bits: 26 bits of Inst. shifted left 2 bits
 - Higher 4 bits: PC+4



Creating a Single Datapath from the Parts

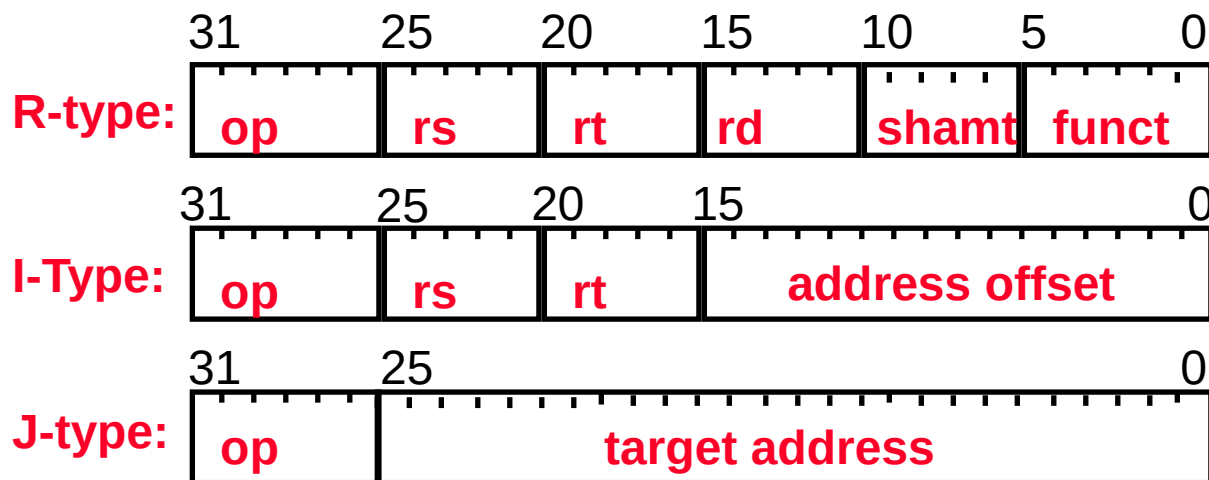
- Assemble the **datapath** segments and add **control** lines
- **Single cycle** design – fetch, decode and execute each instructions in **one** clock cycle
 - **no datapath resource sharing** in the same time, (e.g., separate Instruction Memory and Data Memory, adders)
 - **multiplexors** needed for the input of shared elements
 - **write control signals** for Register File and Data Memory
- **Cycle time** is determined by length of the longest path

Inst. Datapath Portions

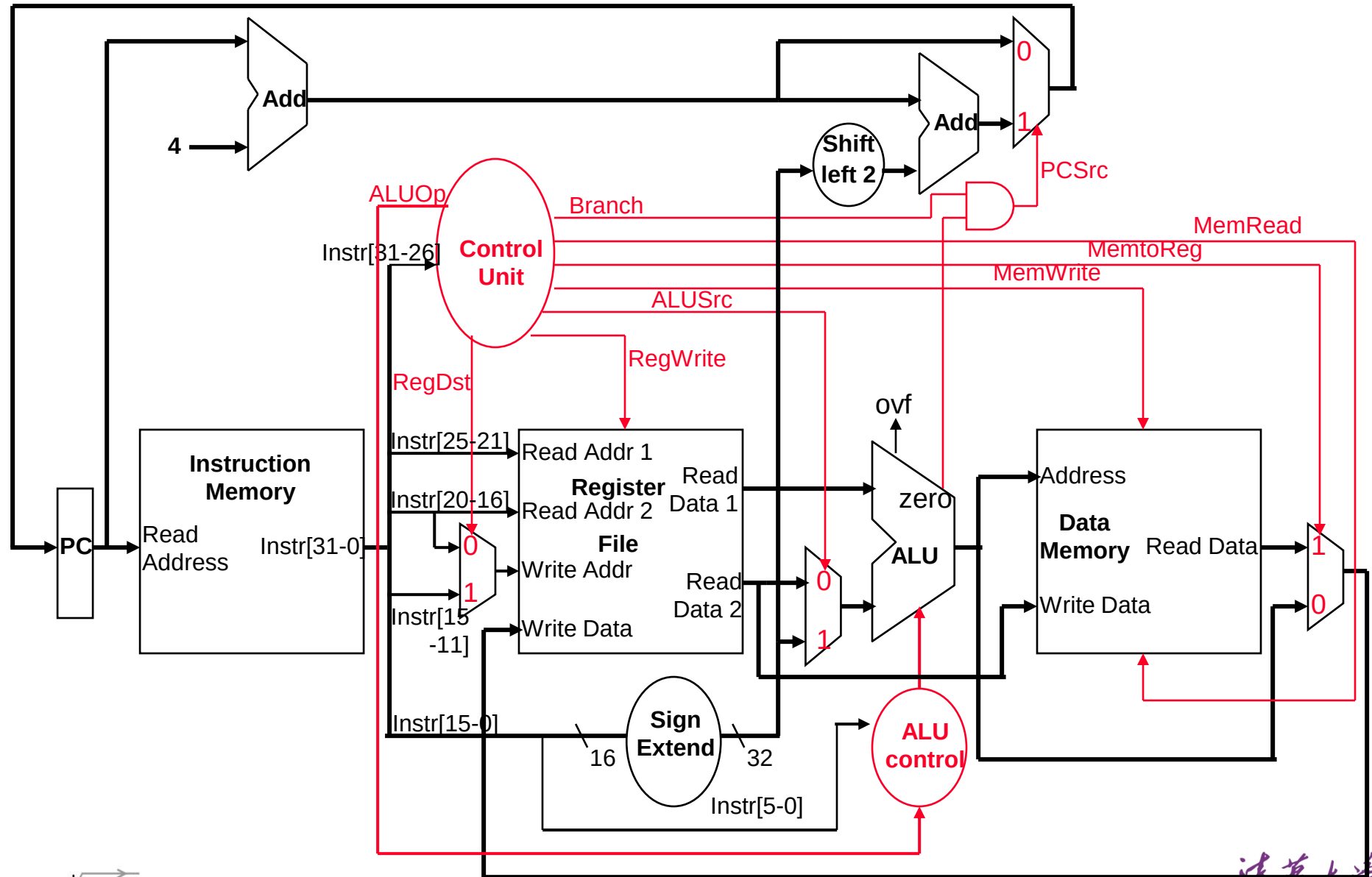


Adding the Control

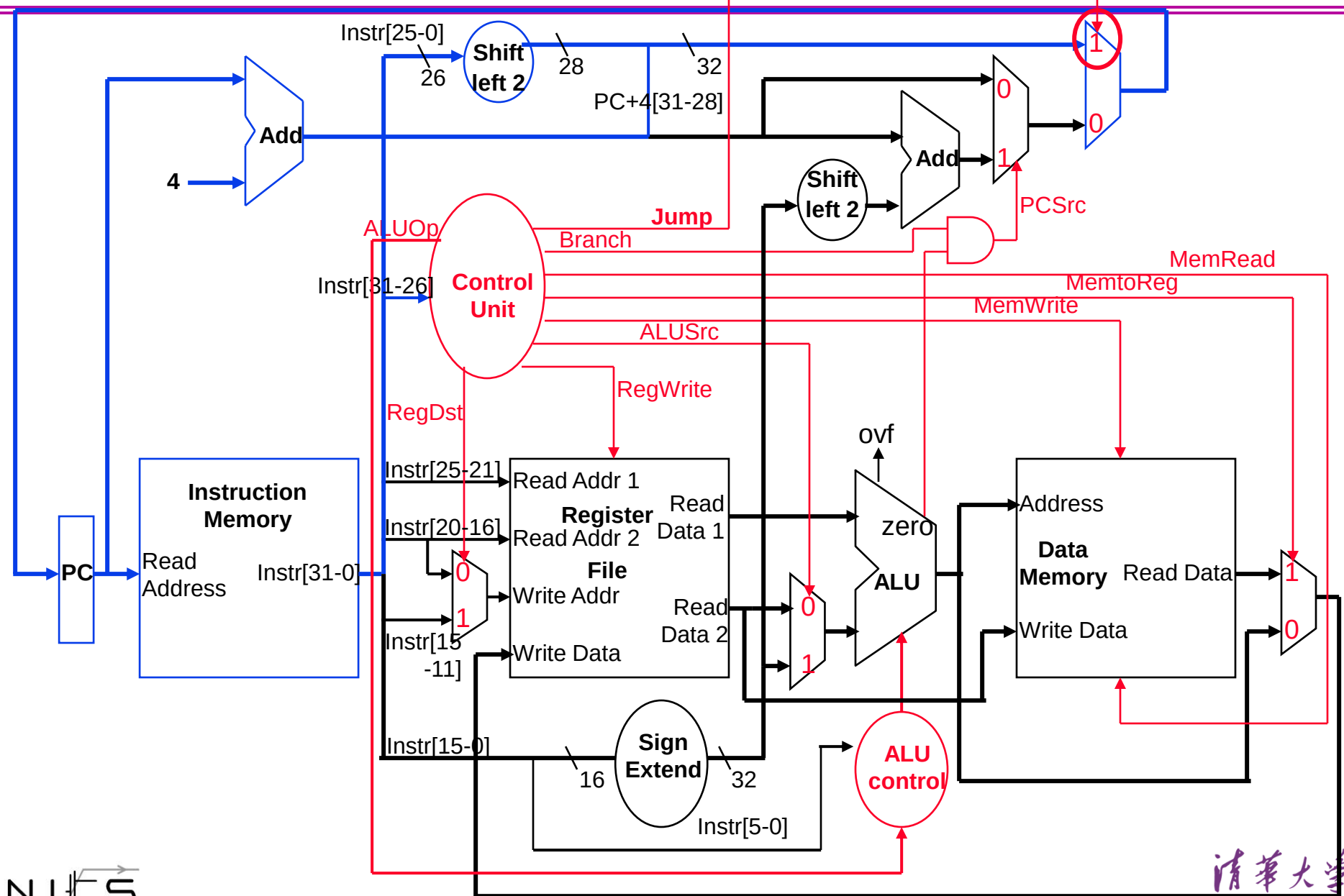
- Selecting the operations to perform (ALU, Register File and Memory read/write)
 - **op field always** in bits 31-26
 - addr of target registers **to be read**: rs field (bits 25-21) & rt field (bits 20-16) for R;
 - rs is the base register for LW and SW
 - addr of target registers **to be written**: rd (bits 15-11) for R Inst.; rt (bits 20-16) for LW;
 - **offset always** in bits 15-0 for J & I
- Controlling the flow of data (multiplexor inputs)



Single Cycle Datapath with Control Unit

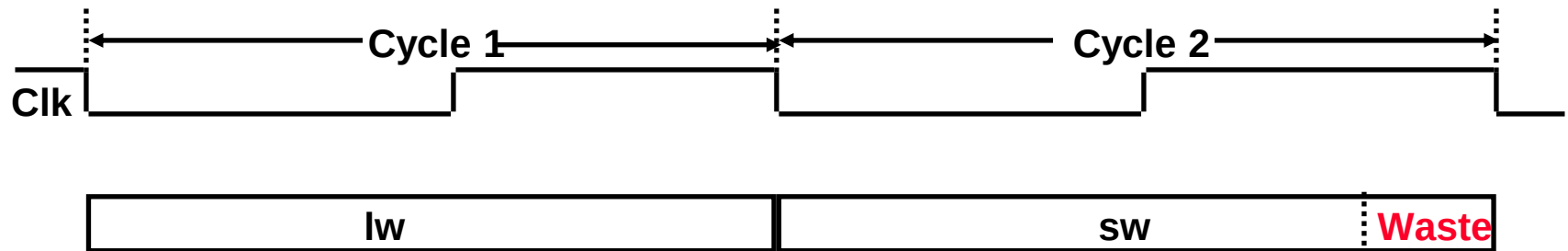


Adding the Jump Operation



Single Cycle Disadvantages & Advantages

- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the **slowest** instruction
 - especially problematic for more complex instructions like floating point multiply



- May be wasteful of area since some functional units (e.g., adders) must be duplicated since they can not be shared during a clock cycle
- Is simple and easy to understand

Outline

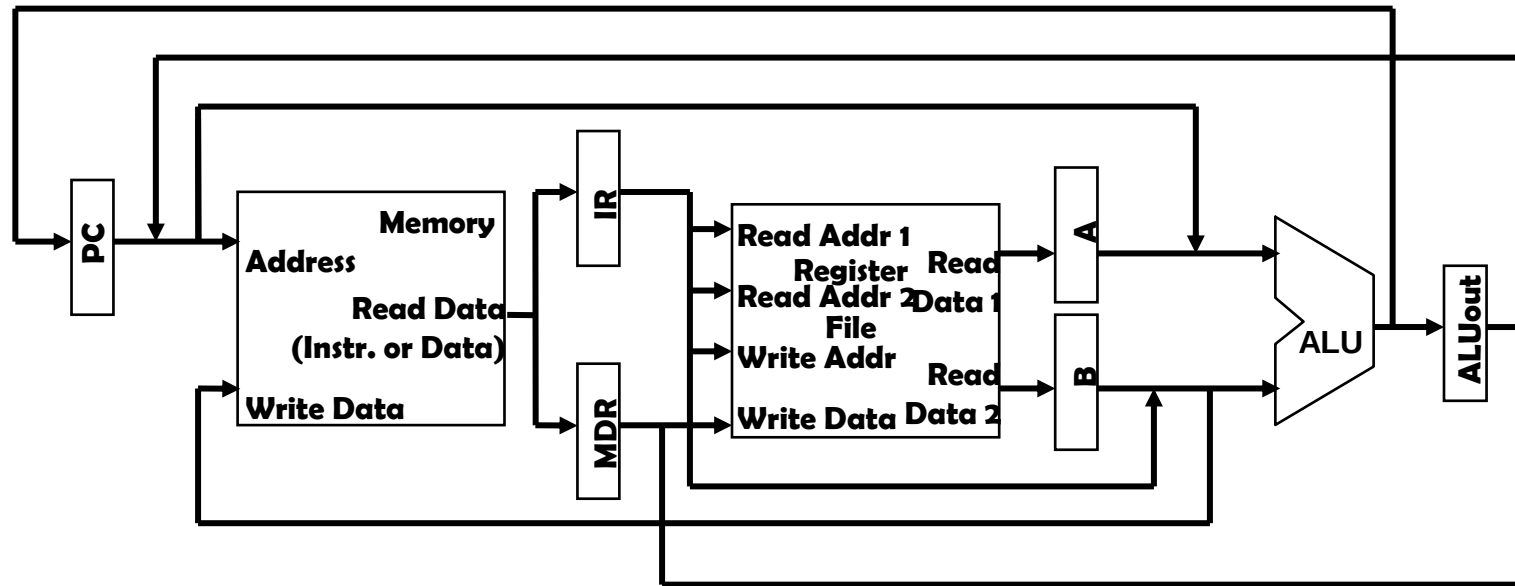
- Review
- Single Cycle Processor
- Multiple Cycle Processor
- Pipeline Processor
- Conclusion

Multicycle Datapath Approach

- Let an instruction take more than 1 clock cycle to complete
 - Break up instructions into several steps
 - Not every instruction takes the *same* number of clock cycles
- **Faster** clock rates
- Functional units that **can be reused** per instruction

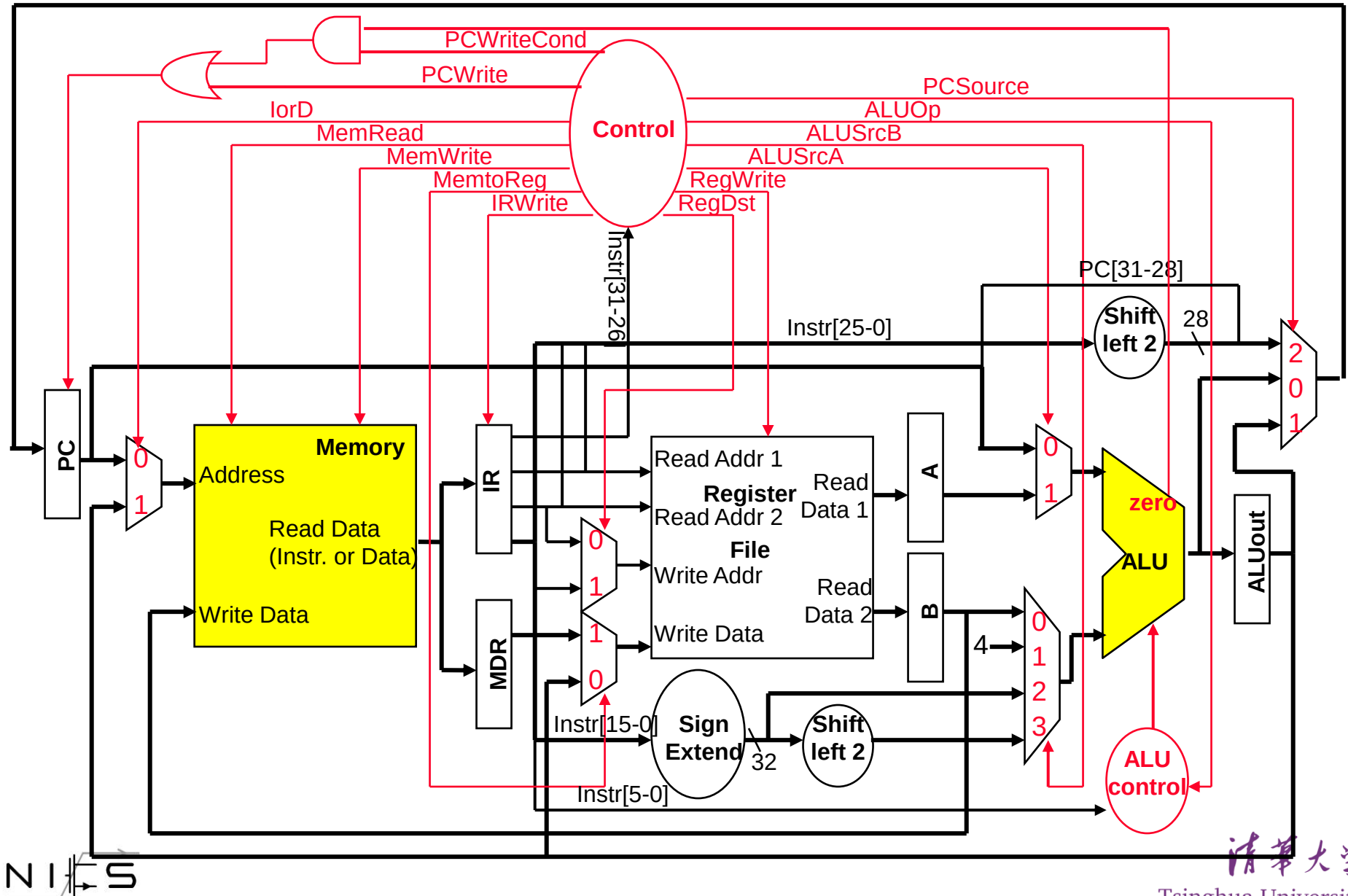
Multicycle Datapath Approach, con't

- At the end of a cycle
 - Store values needed in a later cycle in an internal register



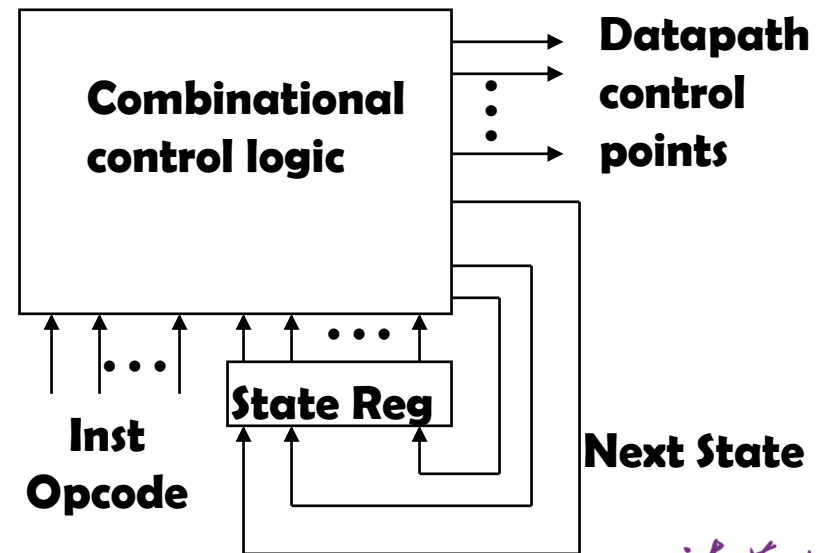
- Data used by **subsequent** instructions are stored in programmer visible registers (i.e., register file, PC, or memory)

The Multicycle Datapath with Control Signals



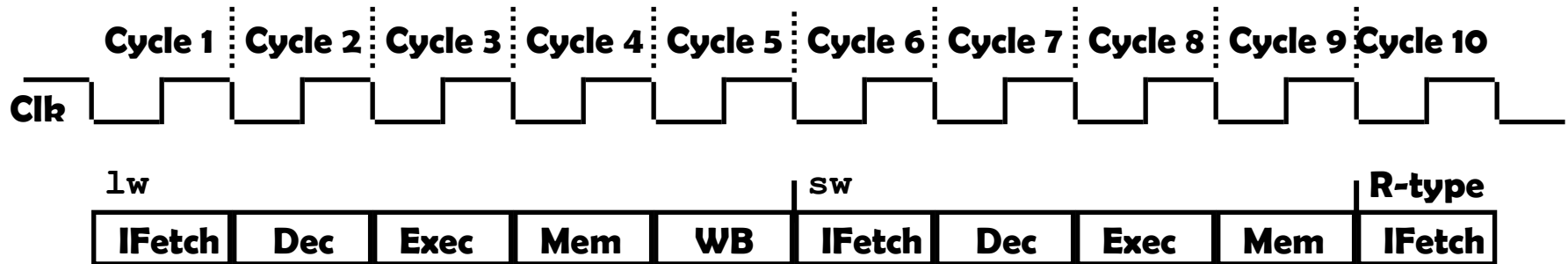
Multicycle Control Unit

- Multicycle datapath control signals are not determined solely by the bits in the instruction
- Must use a finite state machine (FSM) for control
 - a set of states (current state stored in State Register)
 - next state function (determined by current state and the input)
 - output function (determined by current state and the input)



Multicycle Advantages & Disadvantages

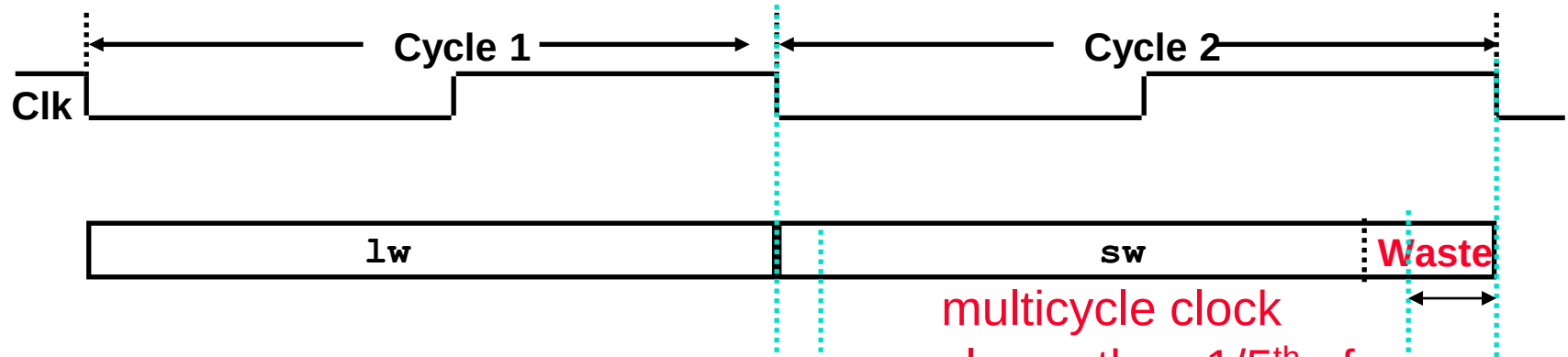
- Uses the clock cycle efficiently – the clock cycle is timed to accommodate the slowest instruction **step**



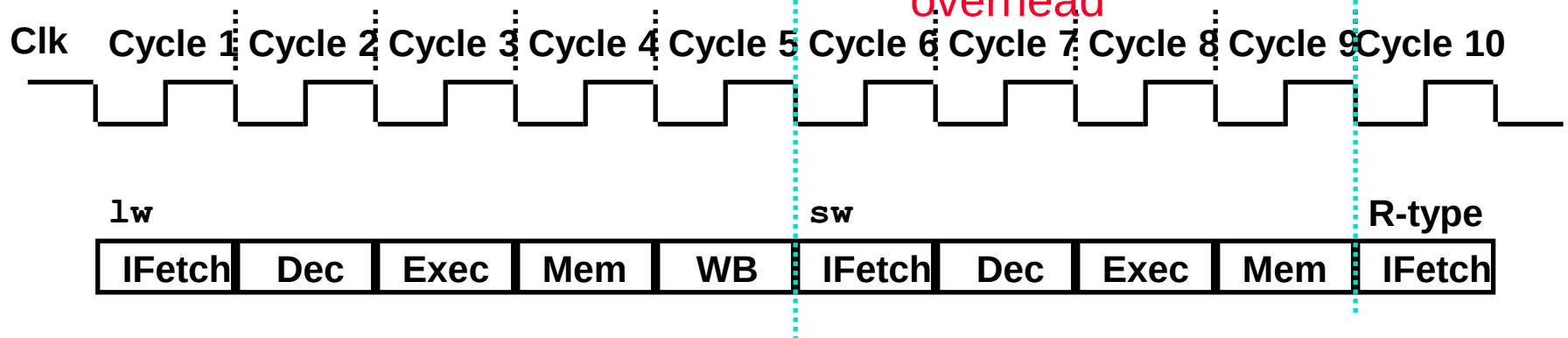
- Multicycle implementations allow functional units to be used more than once per instruction as long as they are used on different clock cycles
- but
- Requires additional internal state registers, more muxes, and more complicated (FSM) control

Single Cycle vs. Multiple Cycle Timing

Single Cycle Implementation:



Multiple Cycle Implementation:

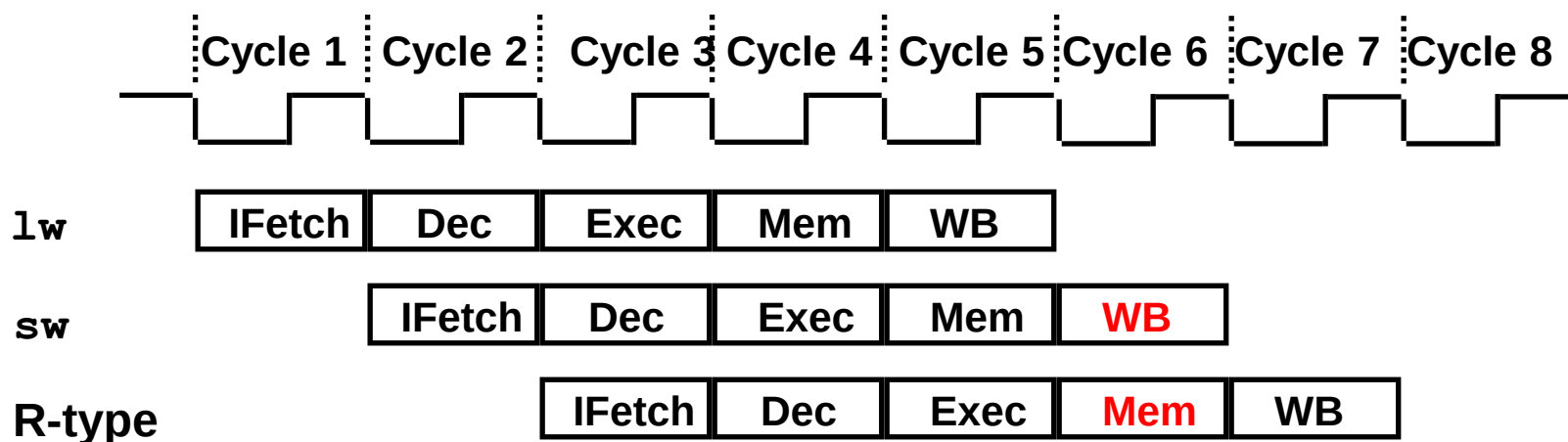


Outline

- Review
- Single Cycle Processor
- Multiple Cycle Processor
- Pipeline Processor
- Conclusion

A Pipelined MIPS Processor

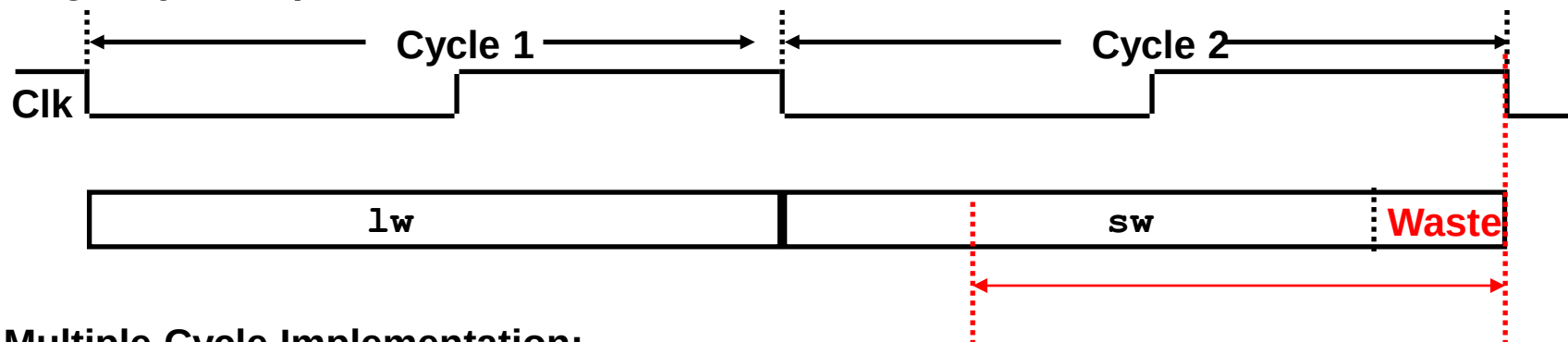
- Start the **next** instruction before the current one has completed
 - throughput** - total amount of work done in a given time
 - latency** (execution time, delay time, response time - time from the start of an instruction to its completion)



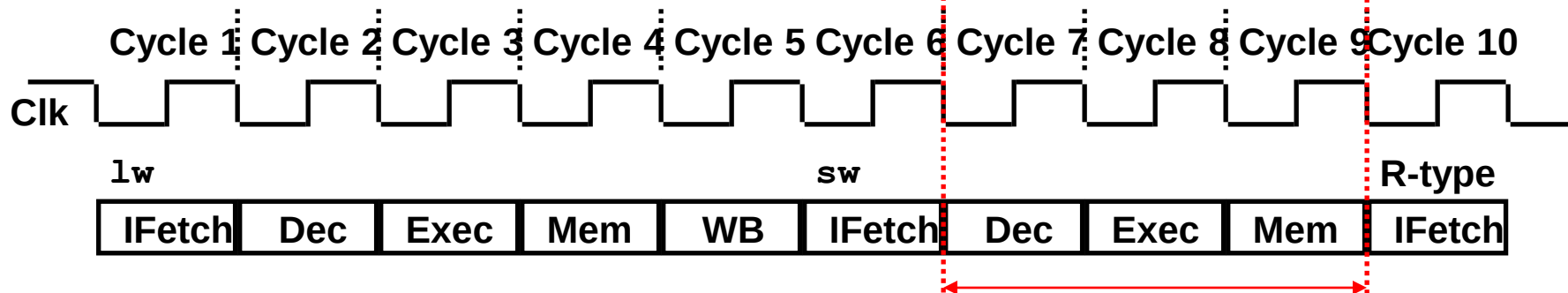
- clock cycle (pipeline stage time) is limited by the slowest stage
- for some instructions, some stages are **wasted** cycles

Single Cycle, Multiple Cycle, vs. Pipeline

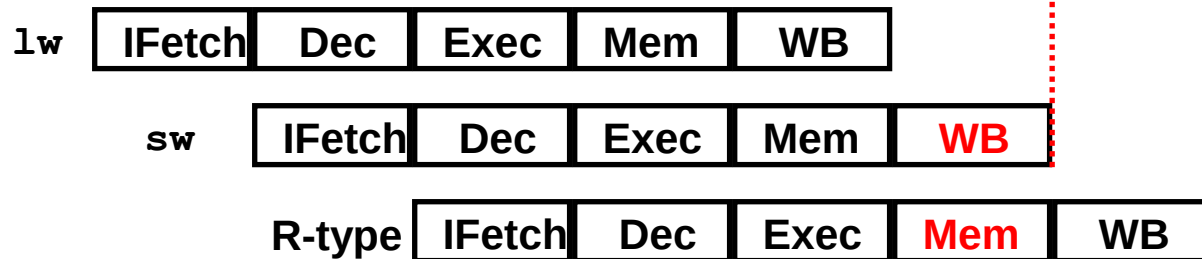
Single Cycle Implementation:



Multiple Cycle Implementation:

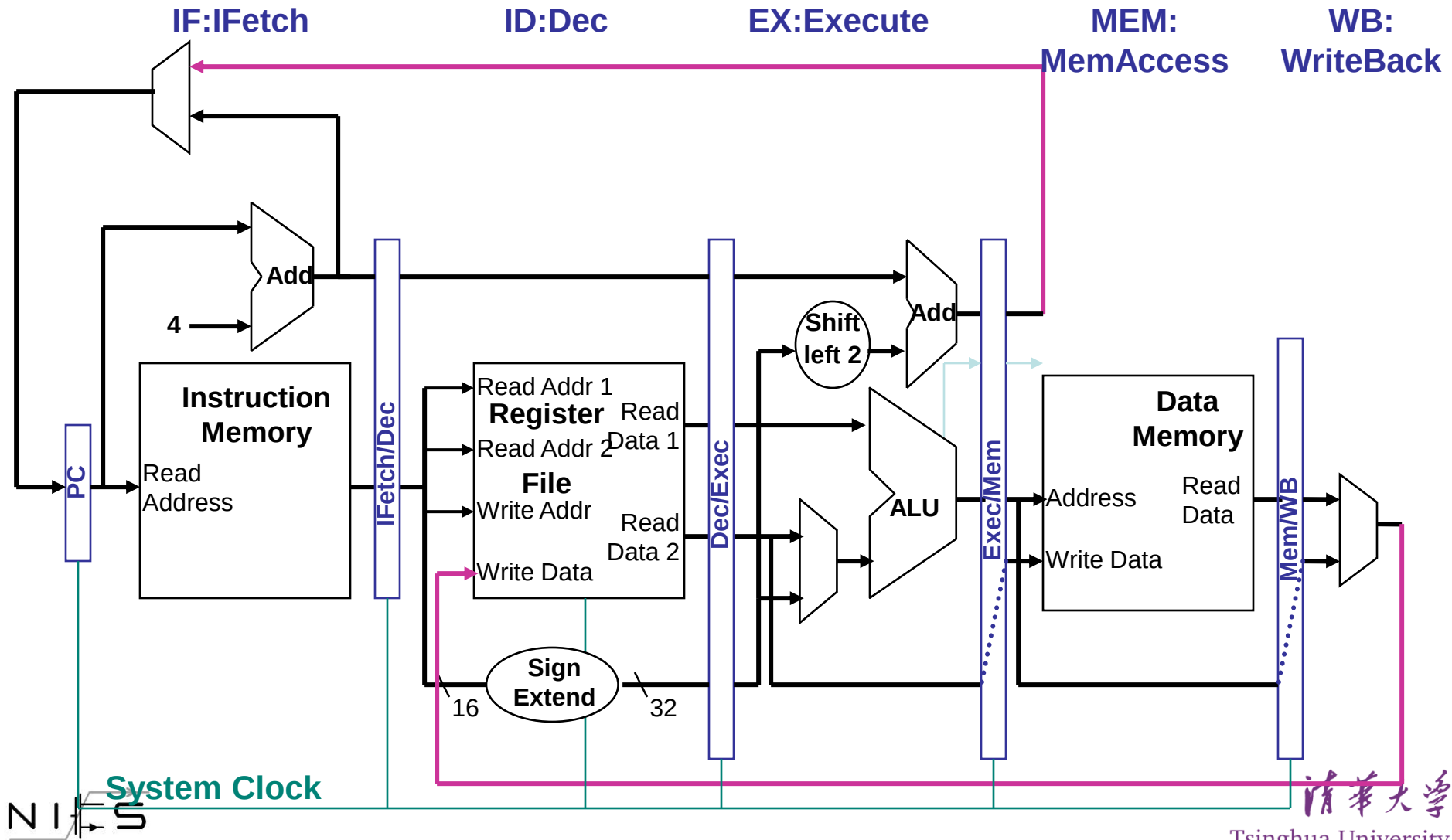


Pipeline Implementation:



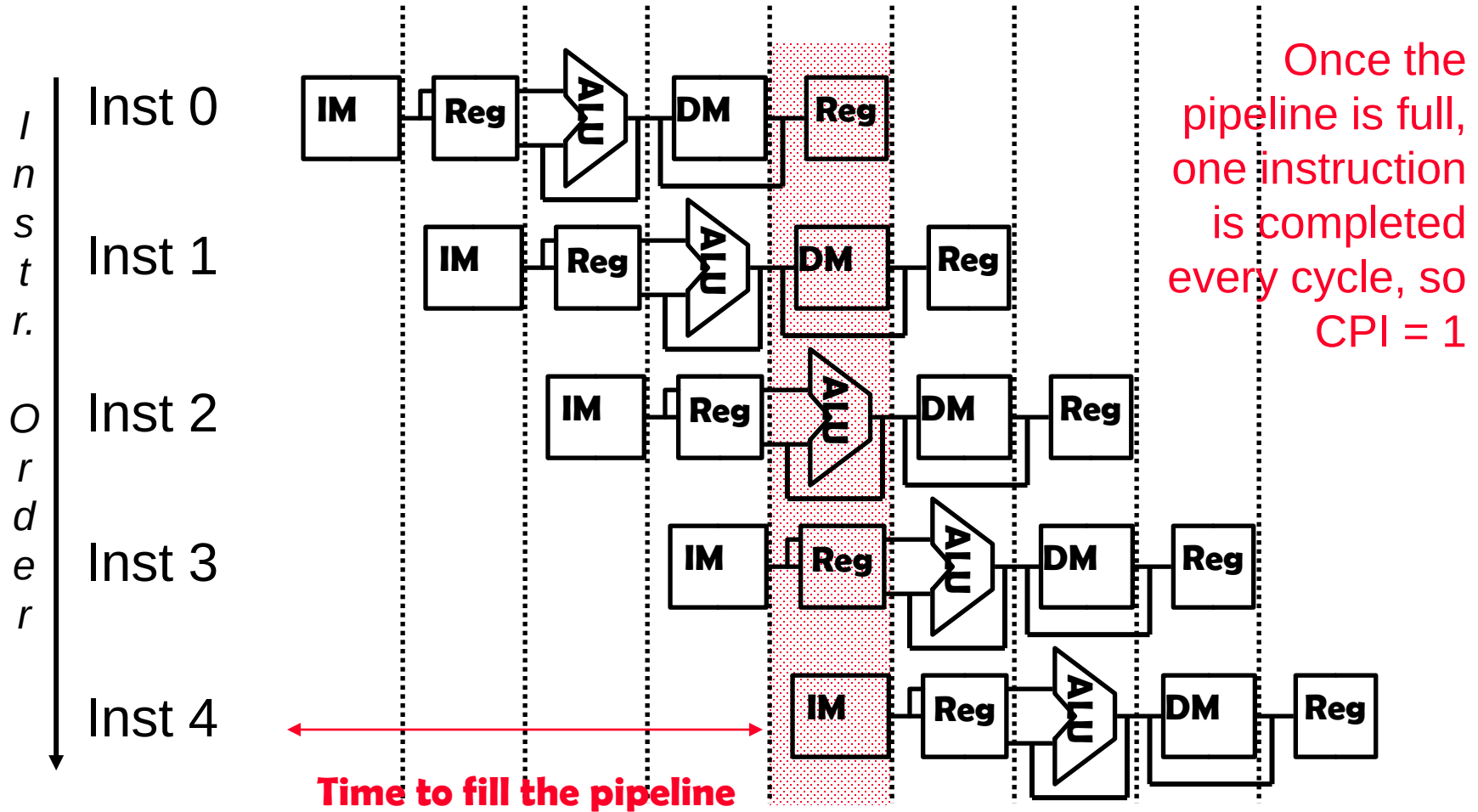
MIPS Pipeline Datapath Modifications

- What do we need to add/modify in our MIPS datapath?
 - State registers between each pipeline stage to **isolate** them



Why Pipeline? For Performance!

Time (clock cycles)



Can Pipelining Get Us Into Trouble?

- Pipeline Hazards

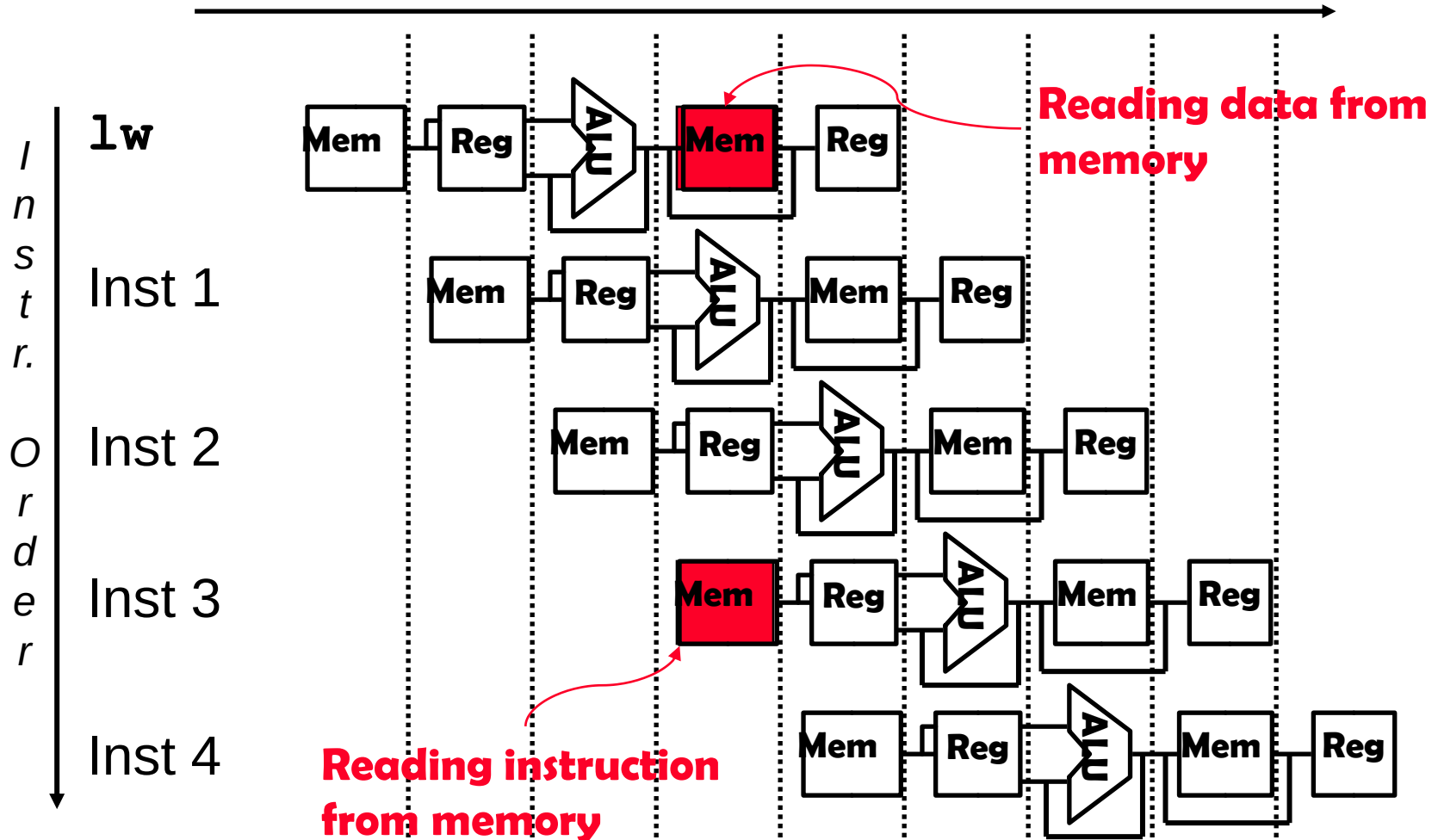
- **structural hazards**: attempt to use the same resource by two different instructions at the same time
- **data hazards**: attempt to use data before it is ready
 - An instruction's source operand(s) are not ready
- **control hazards**: attempt to make a decision about program control flow before the condition has been evaluated and the new PC target address calculated
 - branch instructions

- Can always resolve hazards by **waiting**

- pipeline control must detect the hazard
- and take action to resolve hazards

A Single Memory Would Be a Structural Hazard

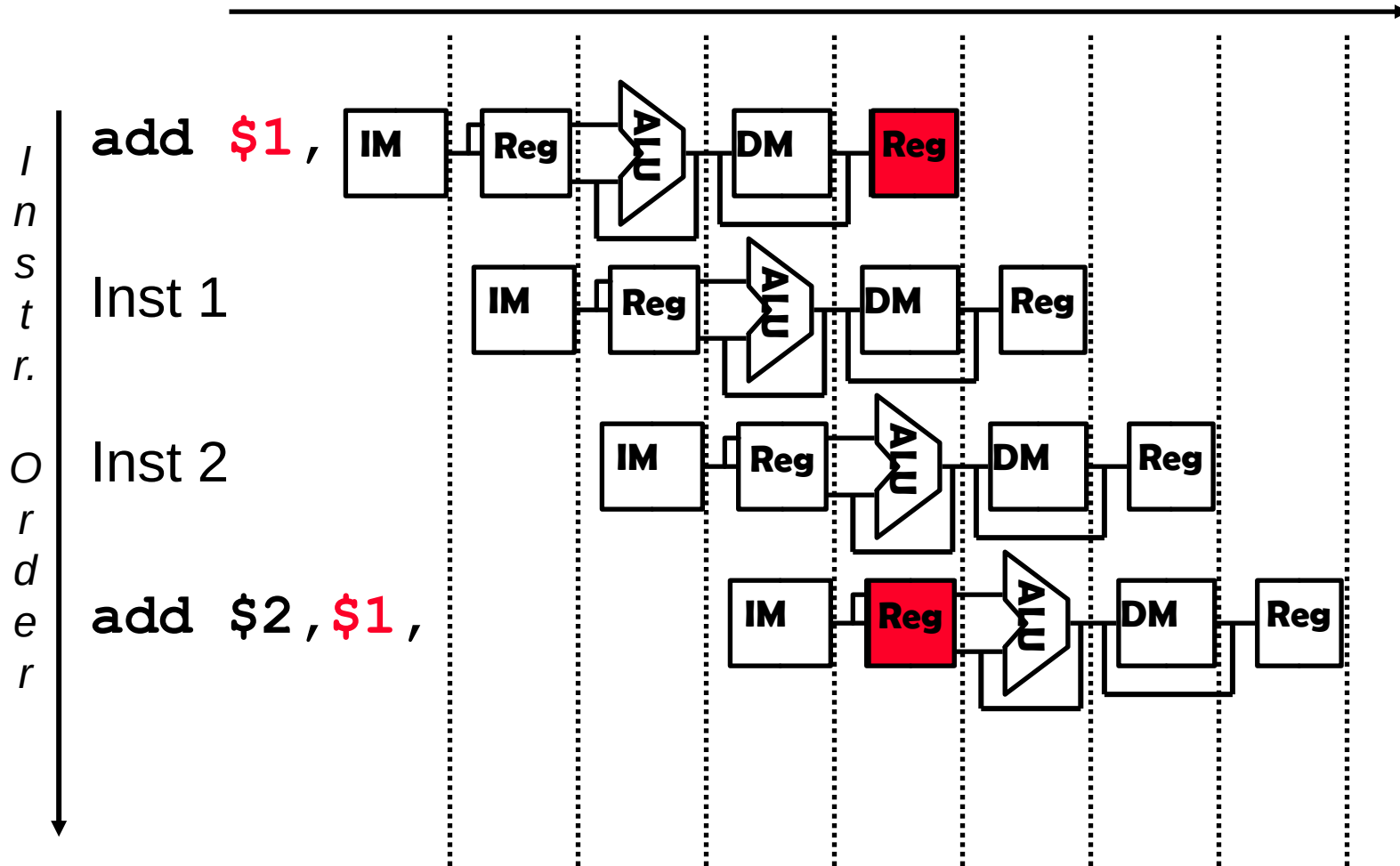
Time (clock cycles)



□ Fix with separate instr and data memories (I\$ and D\$)

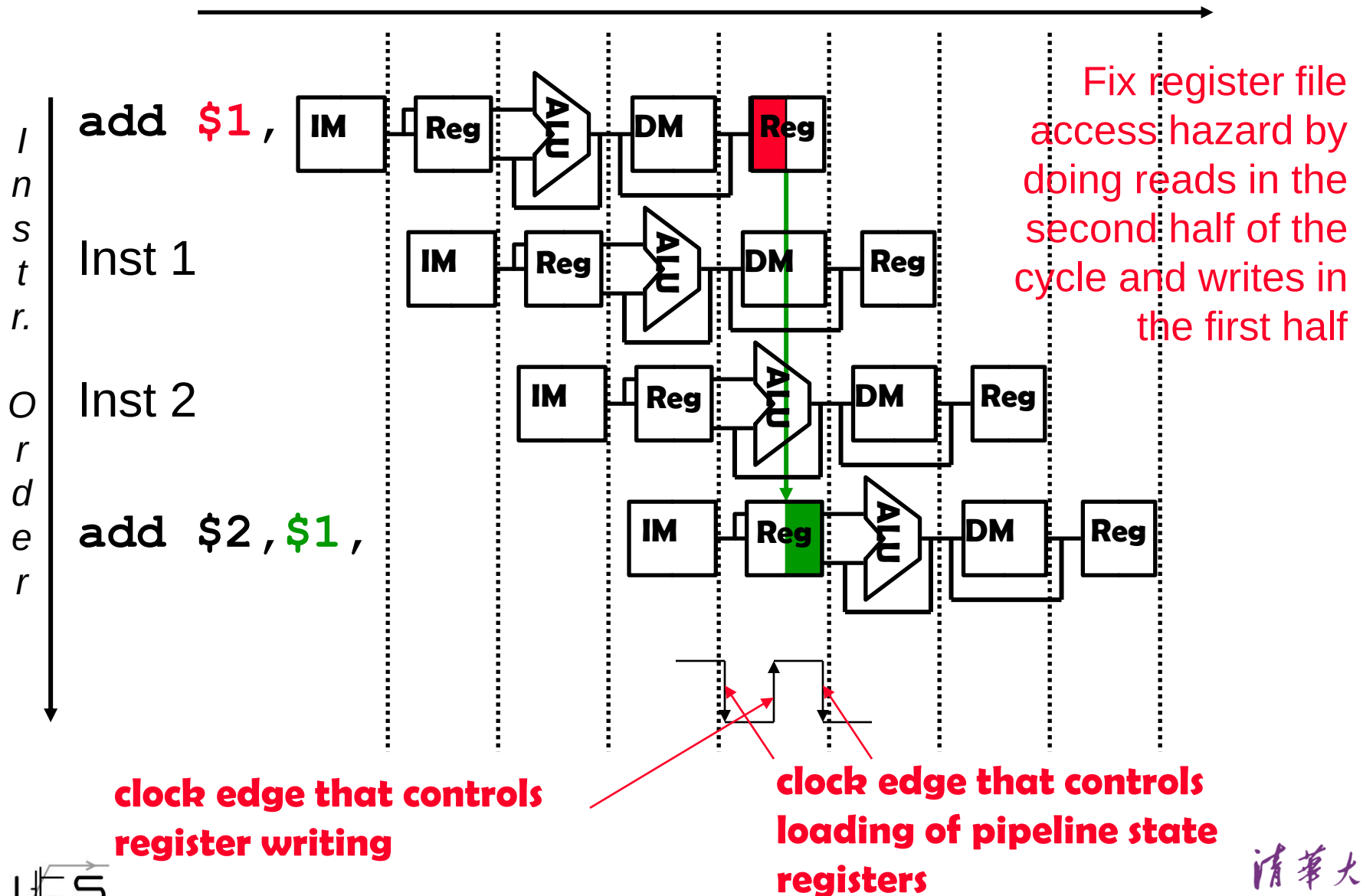
How About Register File Access?

Time (clock cycles)



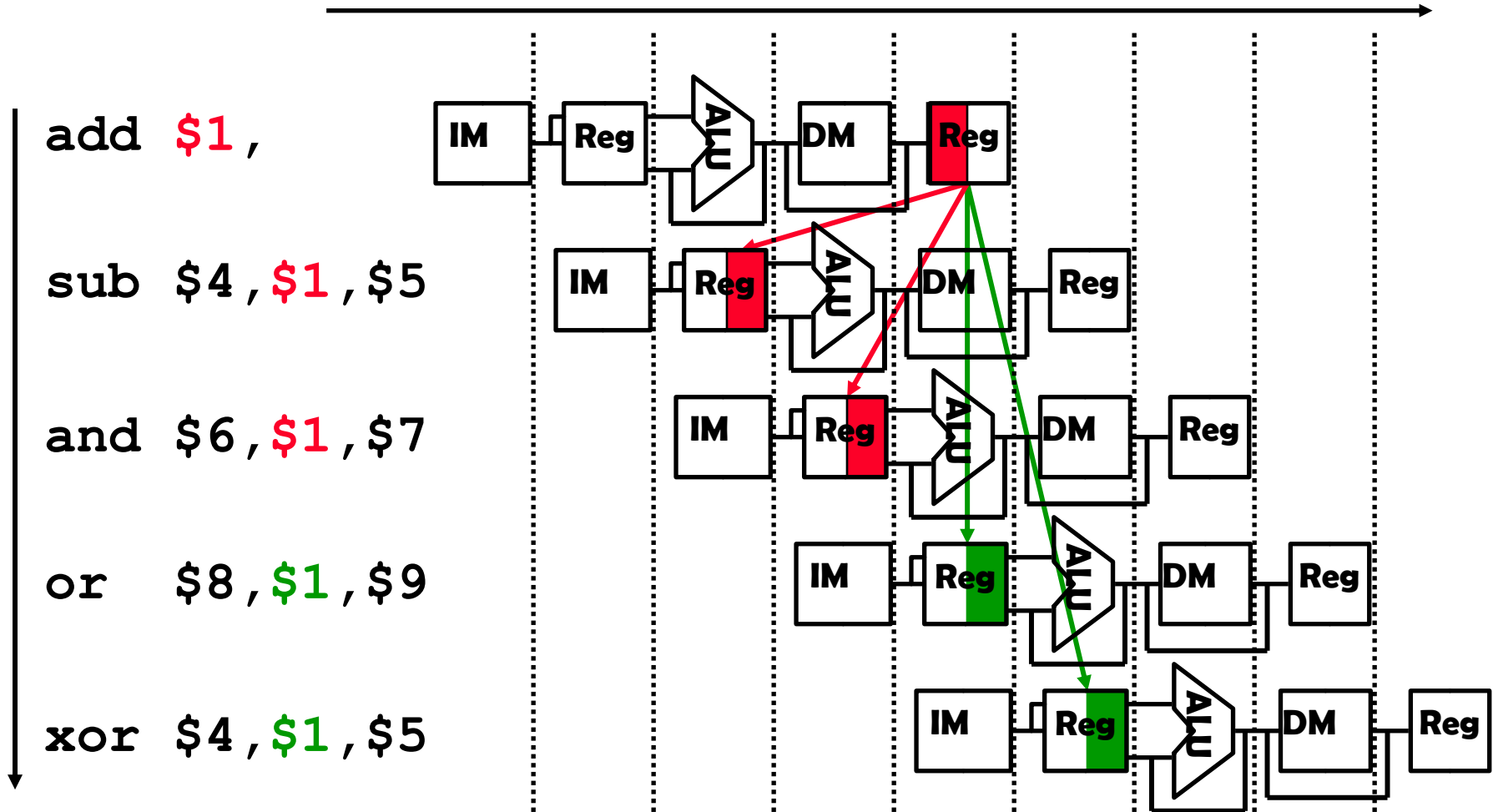
How About Register File Access?

Time (clock cycles)



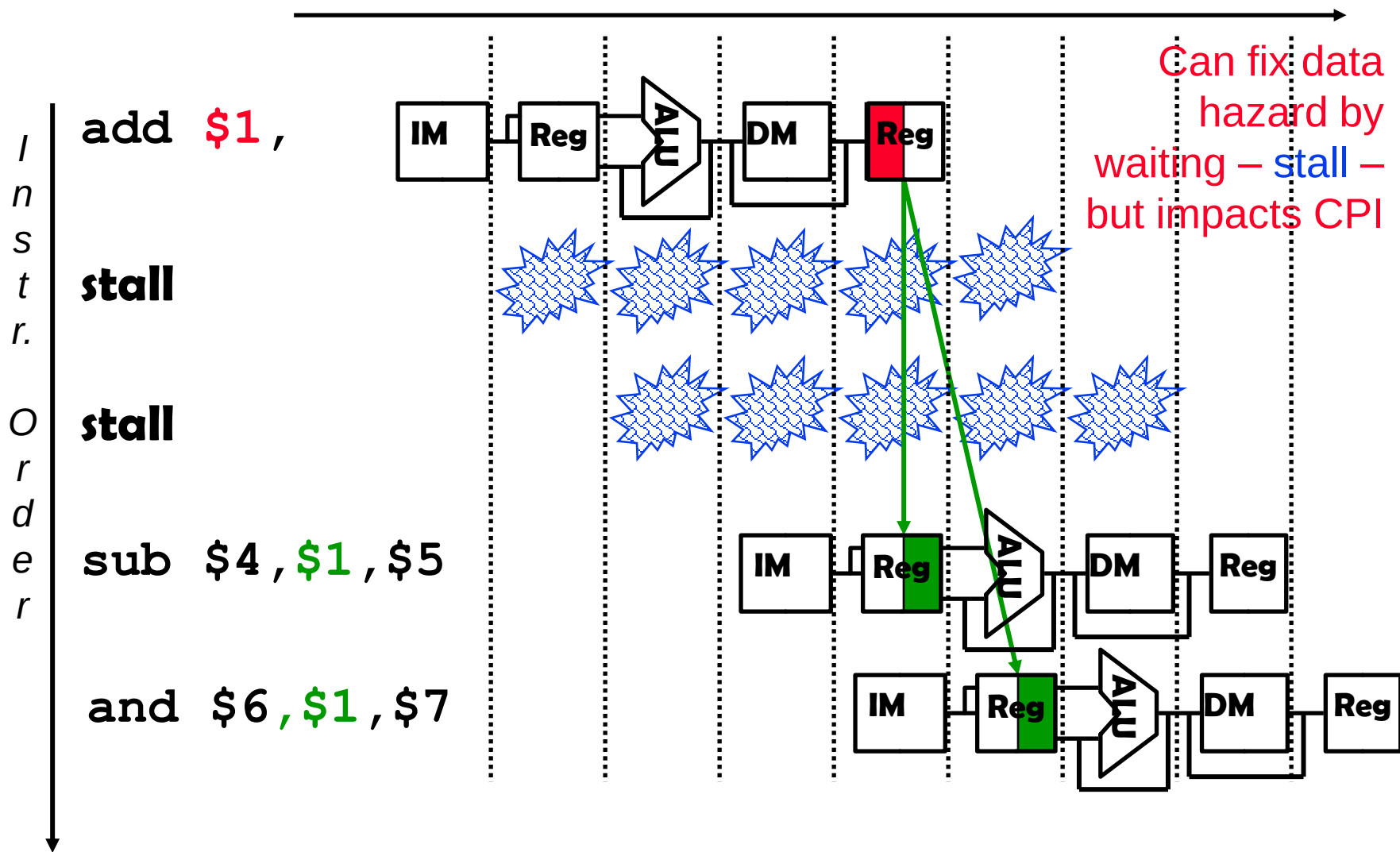
Register Usage Can Cause Data Hazards

- Dependencies backward in time cause **hazards**

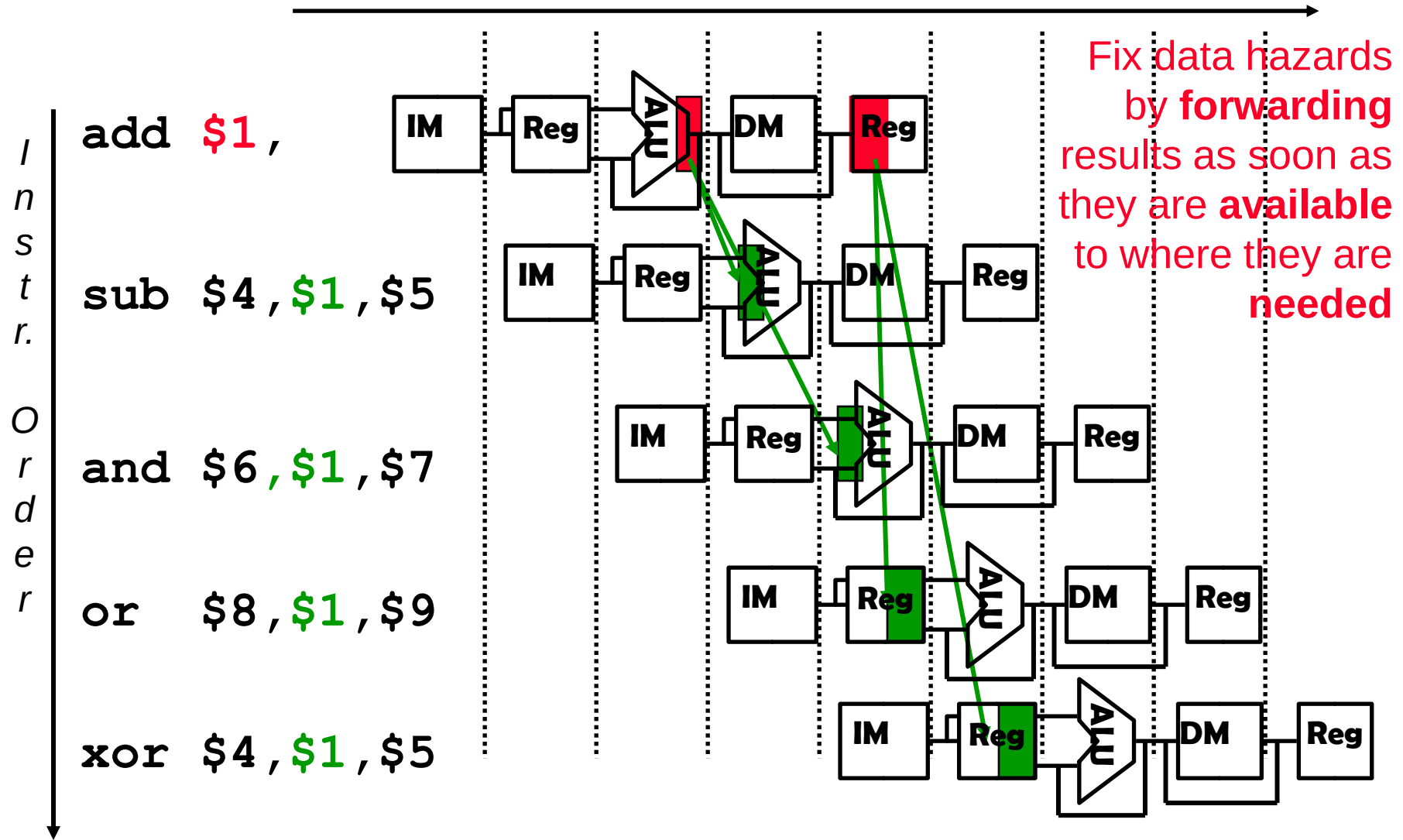


□ Read before write data hazard

One Way to “Fix” a Data Hazard

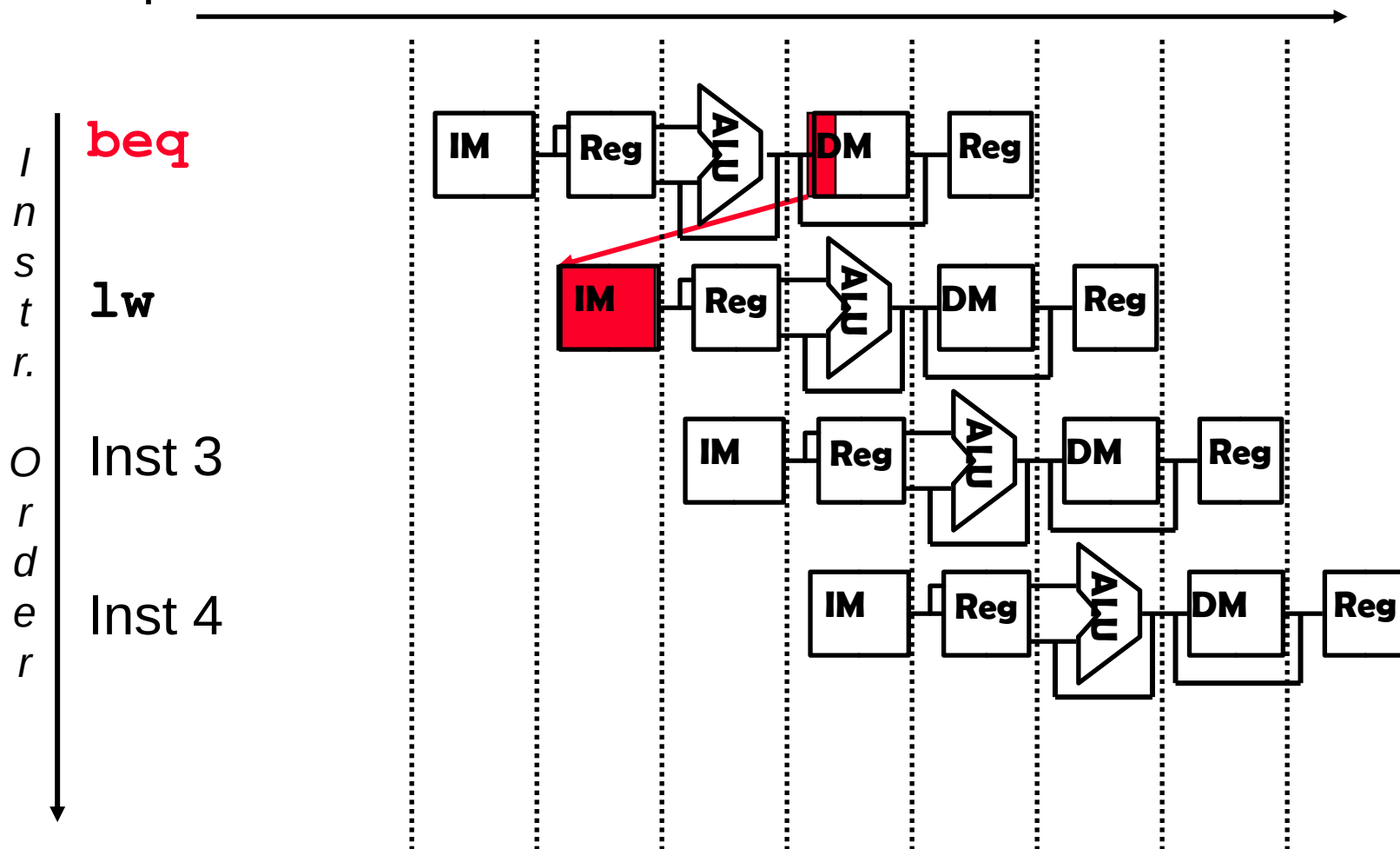


Another Way to “Fix” a Data Hazard

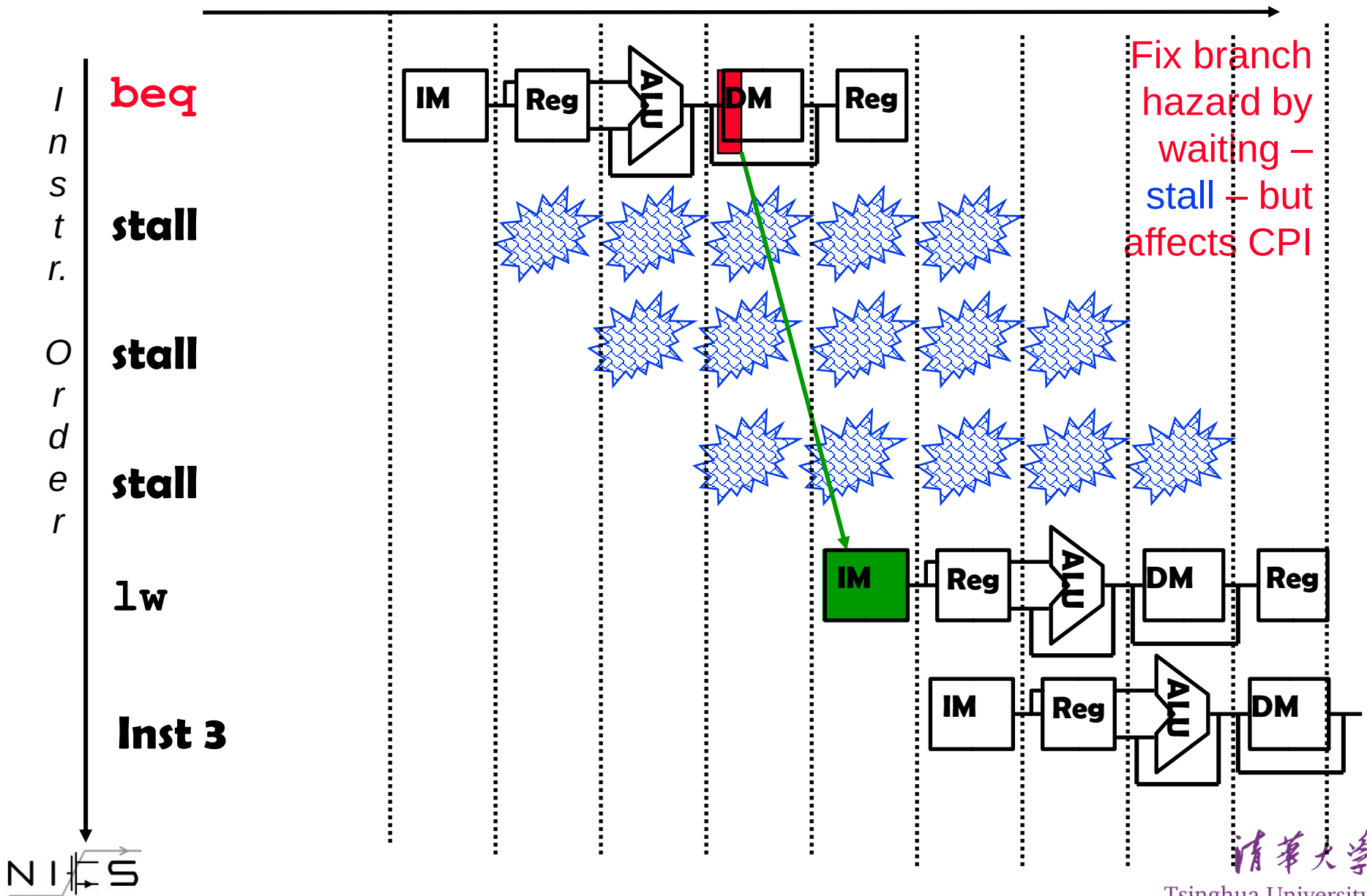


Branch Instructions Cause Control Hazards

- Dependencies backward in time cause **hazards**

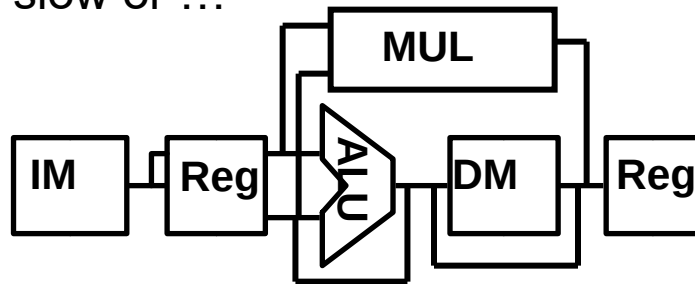


One Way to “Fix” a Control Hazard

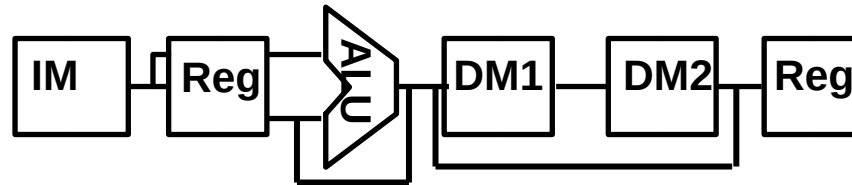


Other Pipeline Structures Are Possible

- What about the (slow) multiply operation?
 - Make the clock twice as slow or ...

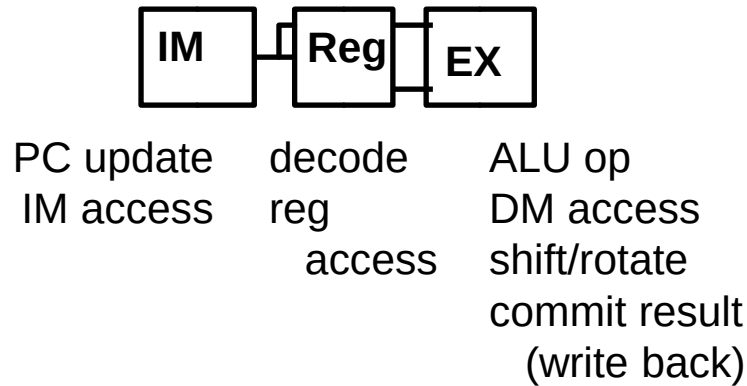


- What if the data memory access is twice as slow as the instruction memory?
 - make the clock twice as slow or ...

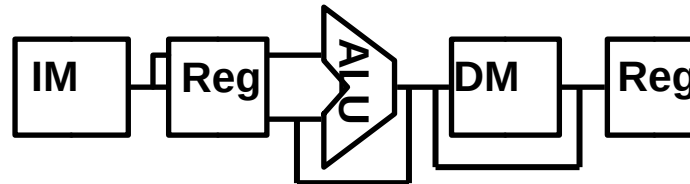


Sample Pipeline Alternatives

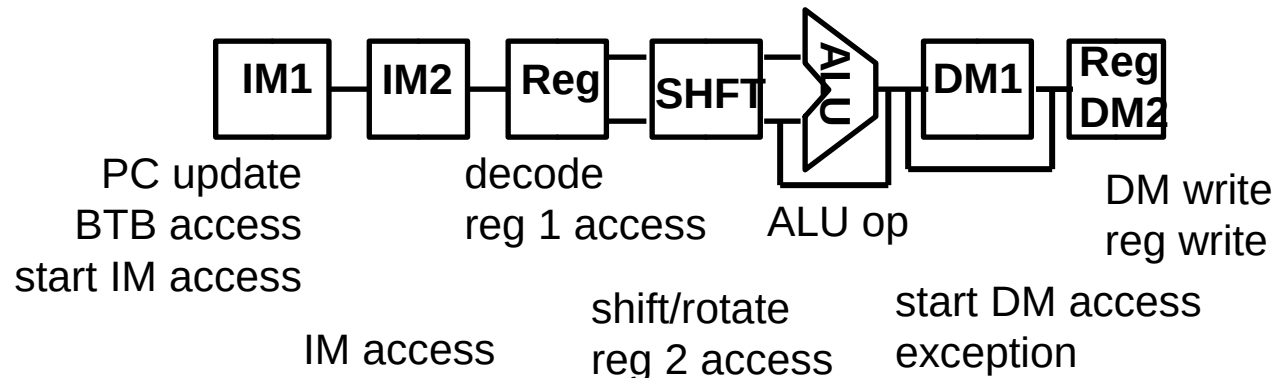
- ARM7



- StrongARM-1



- XScale



Summary

- All modern-day processors use pipelining
- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Potential speedup: a CPI of 1
- Pipeline rate limited by **slowest** pipeline stage
 - Unbalanced pipe stages makes for inefficiencies
 - The time to “**fill**” pipeline and time to “**drain**” it can impact speedup for deep pipelines and short code runs
- Must detect and resolve hazards
 - Stalling negatively affects CPI (makes CPI less than the ideal of 1)